Deep Prompt, Shallow Decoder: Spending Loops on Prefill in Looped Language Models

Yang Hu

Abstract

Looped language models run a block of layers several times for every token and use the same number of loops to read the prompt (prefill) and to generate each answer token (decode). The two phases take very different amounts of time: on Ouro-2.6B-Thinking, prefill is under 0.4% of the time spent on a GSM8K or MATH-500 answer. We ask whether such models should spend more loops on the prompt and fewer on each answer token. We measure where inference time goes on tasks with short and long inputs and outputs, then adapt a looped model so that a decoder with n loops can read the prompt states of any prefill depth m, and vary m from 1 to 16 with one and two decode loops. On Ouro-2.6B with one decode loop, four prefill loops instead of one raise accuracy on four chain-of-thought benchmarks from 31.2 to 41.2 for 16% more time per request, on the same requests and weights; over 11 benchmarks the average rises from 47.2 to 56.8. On most benchmarks the gain stops at two to four prefill loops, also for an adapter trained on prefill depths up to 16. At the same number of decode loops the adapted model beats the original model and models distilled to that depth on the same data, and at one decode loop it also wins at equal measured time (41.2 against 27.0 in about 11 seconds per request). A second decode loop buys more, though: the original model with two loops scores 56.8 in 15.7 seconds, and with two decode loops deeper prefill and distillation reach about the same accuracy in the same time. Deeper prefill does not help few-shot MATH or the exact-recall part of HELMET. Code, results and per-item outputs: `https://github.com/yanghu819/untying-prefill-decode`.

1 Introduction

Looped language models such as Ouro (Dehghani et al., 2019; Zhu et al., 2025) reuse a block of layers several times for each token; Ouro-2.6B runs its block four times. The same four loops are used to process the prompt (prefill) and to generate each answer token (decode), although the two phases cost very different amounts of time. Prefill handles all prompt tokens in one parallel pass, while decoding repeats for every generated token. With long answers, decoding takes almost all of the time: an Ouro-2.6B-Thinking answer to a GSM8K problem takes one to two minutes, and prefill accounts for 0.2–0.3% of it (Section 3).

If prefill is this cheap, a model could read the prompt with more loops than it uses to decode, as a deep encoder with a shallow decoder does in machine translation (Kasai et al., 2021), here within one looped model. The extra prefill loops would add little time, and fewer decode loops would save time on every token. It is not obvious that this works. A decoder with one loop may be unable to use the states of a deeper prefill, and any gain may stop after a few loops.

Running the released model with more prefill than decode loops fails. With four prefill loops and one decode loop, Ouro-2.6B averages 26.6 points over 11 benchmarks, below the 44.2 it reaches with one loop for both (Table 1). We therefore train a small *loop adapter* once (Section 6). It adds LoRA to the decoder and a learned map from each prefill loop's prompt cache to each decode loop, and it is distilled from the original model on all (m, n) pairs up to four loops. With it we measure where a looped model spends its time on each task, how quality and time change as prefill grows from 1 to 16 loops at a fixed decode depth, and how deeper prefill compares at equal measured time with the original model, distilled models and more decode loops. Code, per-request timings and per-item scores are released.

2 Setup

Models and notation. Ouro-2.6B and Ouro-2.6B-Thinking (Zhu et al., 2025) (four loops). "m/n" prefills the prompt with m loops and decodes each answer token with n loops. *Original* n/n is the released checkpoint run with n loops. *Distilled* n/n trains LoRA on the n-loop model to match Original 4/4, with the adapter's data, teacher and 48M answer tokens, at rank 16 and at rank 80 (62.9M parameters, about as many as the adapter's 63.3M); a threefold higher or lower learning rate changes its average by at most 0.6 points (Appendix E). *Ours* m/n is the frozen model with one loop adapter (Section 6); a decoder with n loops reads the prompt states of prefill depth m. The main adapter is trained on $m, n \leq 4$; a second one on $m \in \{1, 2, 3, 4, 6, 8, 12, 16\}$ and $n \leq 4$.

Benchmarks and timing. Base model: 11 benchmarks with the Ouro evaluation protocol (Appendix H), of which we time the four that write chain-of-thought answers: GSM8K, MATH-500, BBH and MMLU-Pro. Thinking model: eight benchmarks including HELMET (Yen et al., 2025) (about 11K-token prompts). Time is measured per request at batch 1 on one B300 GPU in bfloat16 with greedy decoding, split into prefill (to the first token) and decode; the replayed answers are scored with the evaluation's rule, so quality and time come from the same requests. On the base model we time 100 requests per task, the same for every configuration, and average two passes in opposite orders on an otherwise idle machine (Appendix A). Comparisons are paired over items with 95% bootstrap intervals.

3 Where the time goes

Figure 1 splits the time of each request into prefill and decode. Chain-of-thought tasks have short inputs and long outputs. On 192 GSM8K problems, Ouro-2.6B-Thinking reads about 100 tokens and writes about 900, and prefill is 0.2–0.3% of the time; on MATH-500, with 3,000–3,500 output tokens, it is 0.1%. On the base model, whose few-shot prompts are longer (300–1,400 tokens) and answers shorter (100–700 tokens), prefill is 0.3–2.8% of the total time for Original 1/1, 2/2 and 4/4 and Ours 1/1 and 1/2, lowest on MATH-500 and highest on GSM8K and MMLU-Pro. Long prompts with short answers are the opposite case:

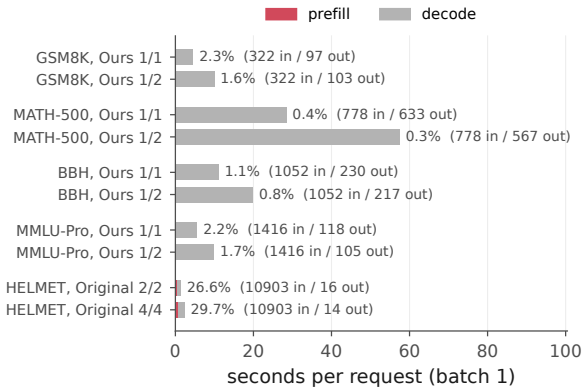


Figure 1: **Decode dominates the time of generation tasks.** Time per request split into prefill (to the first token, red) and decode, batch 1, one B300. Ouro-2.6B with the adapter at prefill 1 (the start of the depth sweep), 100 requests per task; HELMET: Ouro-2.6B-Thinking, Original 2/2 and 4/4, 1,107 requests. Labels: prefill share and mean input / output tokens.

on HELMET (about 10,900 input and 15 output tokens with the chat template) prefill is 27–30% of the time at Original 2/2 and 4/4. Prefill cost grows about linearly with its loops, so a small share p at one loop becomes about $m\,p$ at m loops. Tasks with a small share leave room for a deeper prefill; tasks with long prompts and short answers do not.

4 How far deeper prefill goes

We now fix the decode loops and change only the prefill loops (Figure 2). With the adapter trained on $m \leq 4$ and the same weights throughout, four prefill loops instead of one raise the average over 11 benchmarks from 47.2 to 56.8 at one decode loop and from 57.9 to 67.2 at two (gains of 9.0 ± 0.6 and 9.1 ± 0.2 over three training seeds). Ouro-2.6B-Thinking behaves the same way at two decode loops (GSM8K 90.7 to 94.1 and MMLU-Pro 36.4 to 51.0 from $m = 1$ to $m = 4$, no further gain at $m = 6$).

Beyond four loops accuracy falls. At one decode loop, GSM8K drops from 45.0 at $m = 4$ to 31.9 at $m = 16$ and RULER-16K from 64.7 to 39.9, while MMLU-Pro still rises to 48.1 at $m = 6$ before falling to 31.7. This adapter never saw such deep states, but one trained on prefill depths up to 16 (dashed) shows a similar shape. To place the end of the gain, we pick, on one half of each benchmark, the first depth that none of the next three tested depths beats by more than 0.5 points, and report accuracy on the other half (Appendix B). For the adapter trained up to 16 this stop is at two

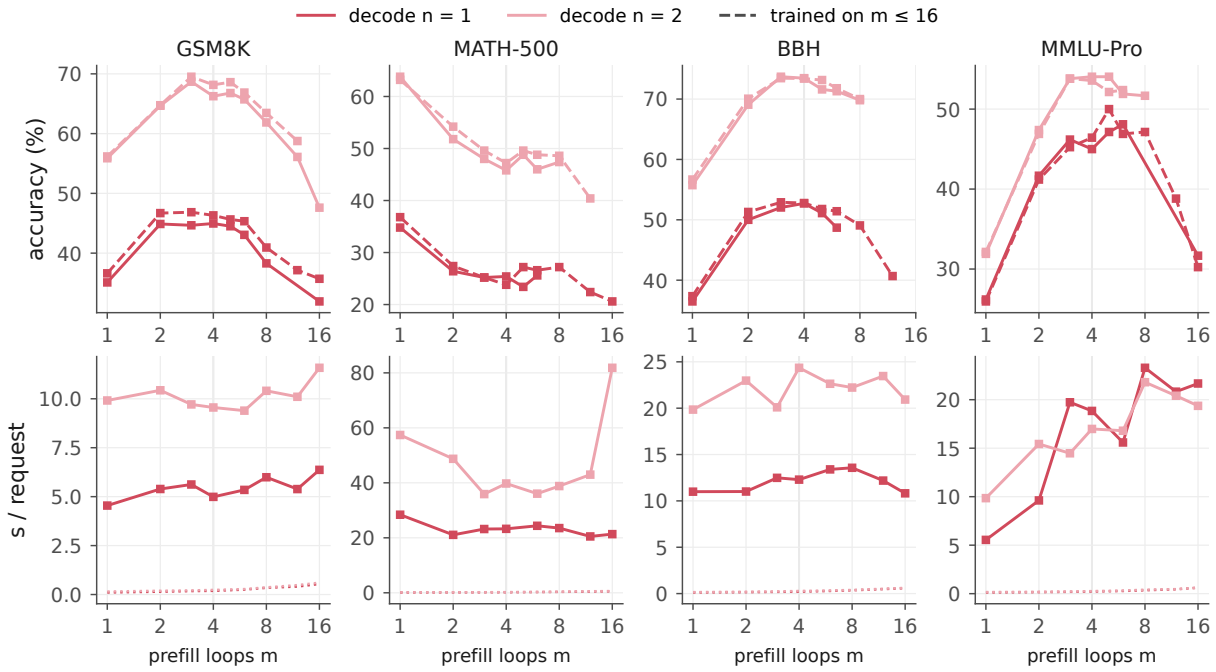


Figure 2: **Prefill depth at fixed decode depth.** Ouro-2.6B, same weights at every m; decode $n = 1$ (dark) and $n = 2$ (light). Top: accuracy on the evaluation sets of Appendix H, solid for the adapter trained on $m \leq 4$ (it extrapolates right of the grey vertical line at $m = 4$), dashed for the one trained on $m \in \{1, 2, 3, 4, 6, 8, 12, 16\}$. Bottom: time per request on 100 timed requests per task, with the maps applied at each decode step (dotted: its prefill part).

or three prefill loops in 11 of the 18 benchmark and decode-depth settings, at four or five in three (MMLU-Pro and HumanEval+ at one decode loop, MBPP+ at two) and at one in both MATH-500 settings; the other two have no stop within the tested depths. On the held-out half of GSM8K at one decode loop, this adapter reaches 47.0 at $m = 2$ and 36.9 at $m = 16$, against 32.3 at $m = 16$ for the adapter trained on $m \leq 4$. The stop marks where the gain ends on these depths, not a proven ceiling. On Ouro-1.4B the best prefill depth is three or four on 13 of 18 settings, with lower accuracy beyond (Appendix C).

With folded maps (Section 6), time per request grows by 16% from $m = 1$ to $m = 4$ at one decode loop, almost all because answers get longer (276 to 319 output tokens on average).

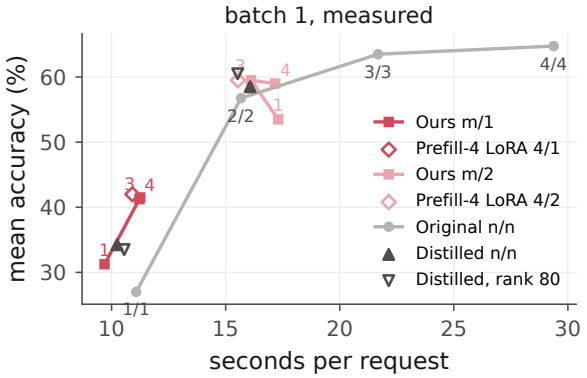


Figure 3: **Quality against measured time per request.** Ouro-2.6B, mean accuracy and mean time per request over the same 100 requests of each of GSM8K, MATH-500, BBH and MMLU-Pro (batch 1, one B300, mean of two passes); Ours with the maps folded into the prompt cache. Labels: prefill loops m or n/n.

5 Is it the best use of the time?

Deeper prefill is worth it only if no other use of the same time does better. At the same number of decode loops, Ours $4/n$ is above Original and Distilled n/n (Table 1), also at rank 80 and without the benchmark items that overlap the training data (Appendix G). A plain LoRA trained only at prefill four, with the maps fixed at the identity, does as well at that depth. What the maps and the training over all (m, n) pairs add is one set of weights for every pair, which the sweep of Section 4 needs, and long-context retrieval at one decode loop (RULER-16K 64.7 against 58.3, and 49.8 when the same all-pairs LoRA is trained without maps).

Equal decode loops do not mean equal time.

	$n=1$	$n=2$
Original n/n	44.2	63.0
Training-free $4/n$	26.6	30.9
Distilled n/n	49.8	65.3
Distilled n/n, rank 80	50.2	65.4
Prefill-4 LoRA $4/n$	57.1	67.3
Ours $4/n$, maps frozen	55.0	66.8
Ours $4/n$, one cache	56.4	66.4
Ours $4/n$, aligned	56.8	67.2

Table 1: **Same decode loops, different prefill.** Ouro-2.6B, average accuracy (%) over 11 benchmarks at one and two decode loops (Original 4/4: 70.2). Each benchmark on the items all rows share; per benchmark: Table 5.

Figure 3 plots accuracy against measured time on the same requests. With the maps folded into the prompt cache, Ours decodes within 10% of the original model's time per token. On the four chain-of-thought tasks, Ours 4/1 scores 41.2 in 11.2 s per request, against 27.0 in 11.1 s for Original 1/1 and 34.2 in 10.2 s for Distilled 1/1 (prefill-4 LoRA: 42.0 in 10.9 s; paired over the 400 requests: $+14.2$ [9.0, 19.8] and $+7.0$ [1.8, 12.5] points), but Original 2/2 scores 56.8, $+15.5$ [10.2, 20.8] over Ours 4/1 in 1.40 times its time. At two decode loops, Ours 4/2 (59.0 in 17.2 s; 3/2: 59.5 in 16.1 s), Distilled 2/2 (58.5 in 16.1 s; rank 80: 60.5 in 15.5 s) and the prefill-4 LoRA (59.5 in 15.5 s) show no detectable difference on these 400 requests (paired differences -1.5 to $+0.5$, every interval containing zero). At batch 32 (estimated, per-step maps) a second decode loop still buys more (Figure 4). Long prompts with short answers lead to the same conclusion: on 1,107 HELMET requests (Ouro-2.6B-Thinking), Ours 4/2 scores 59.1 in 2.10 s, above Original and Distilled 2/2 (44.0 and 54.2), but Original 4/4 reaches 65.7 in 2.33 s.

6 Method

In the original model, a decoder with n loops only ever reads prompt states produced by the same n loops. After a deeper prefill, the prompt keys and values come from loops the decoder has never seen, and the frozen model fails (Table 1, training-free). The loop adapter (Figure 10 in Appendix E) has three parts: rank-16 LoRA (Hu et al., 2022) on the decoder's attention projections, and a key map and a value map for each decode loop, layer and KV head, which transform the prompt cache that loop reads. The key map is one complex scale per rotary frequency pair; it commutes with RoPE (Su et al., 2024), so it can be applied to the query. The value map is linear and is applied to the attention output. Decode loop k reads prefill loop $\max(m-n+k, 1)$, so at $m=n$ every decode loop reads the cache the original model would read. Both maps can also be folded into the prompt cache once at prefill, one mapped copy per decode loop; decoding then runs the original model's attention, and we time this form. A one-cache variant (decode loops read only the last prefill loop) saves memory at slightly lower accuracy (Table 1). The adapter (63M parameters, 2.4% of the model) is distilled once from Original 4/4: KL on answer tokens over 48M tokens of reasoning, instruction, long-context and web data, one (m, n) pair per micro-batch (Appendix G).

Ablations and limits. Training the same adapter with the maps frozen at the identity changes ten of the 11 benchmarks by at most 2.8 points at 4/1 but costs 14.9 to 22.8 on RULER-16K over three seeds (Table 5), while at 4K and 8K tokens the two differ by at most 1.4 points: the maps matter for the longest prompts. Deeper prefill does not help everywhere. On 4-shot MATH-500 one prefill loop is best: 34.8 against 25.4 for four at one decode loop, and 63.8 against 45.8 at two. Without the exemplars four prefill loops are better, by 8.0 [4.0, 11.8] points at one decode loop and 10.4 [6.6, 14.2] at two (Appendix R). Exact recall in HELMET does not improve either (Appendix R).

7 Related Work

Looped models (Dehghani et al., 2019; Geiping et al., 2025; Zhu et al., 2025; Bae et al., 2025a,b) reuse layers; Ouro, ELR and Dory reuse one loop's KV (Zhu et al., 2025; Ladhak et al., 2026; Fishman et al., 2026). Early exit (Schuster et al., 2022; Elhoushi et al., 2024) and Continuous Depth Batching (Schwethelm et al., 2026) vary depth per token, and LoopDuet (Du et al., 2026) (concurrent) sets the two loop budgets of Huginn without training. Deep encoders with shallow decoders (Kasai et al., 2021; Bergner et al., 2024; S et al., 2024) put more depth on the prompt across separate networks; SwiftKV (Qiao et al., 2025) puts less. More in Appendix F.

8 Conclusion

On the frozen Ouro checkpoints we study, generation is almost all decoding, and deeper prefill with a loop adapter raises accuracy at a fixed decode

depth up to two to four loops, beats distillation and leads at equal time with one decode loop; on long chain-of-thought a second decode loop buys more.

Limitations

Time is measured at batch 1 on one GPU type; at large batch sizes decode is cheaper per token and the prefill share grows. Applied at every decode step, the adapter's maps and a second attention call cost 19–40% more decode time per generated token than the original model at batch 1; folding the maps into the prompt cache removes this cost but stores one prompt cache per decode loop, as the original model does at n loops. The depth sweep and the batch-32 estimate use the per-step form. The prefill depth is fixed per setting, not routed per prompt. The prefill loops stay frozen, so the gain might stop later if they were trained for deeper use as well. Deeper prefill does not help few-shot MATH-500 or exact recall in HELMET (34 items). Models are 2.6B with four loops (1.4B, a 9B hybrid and decision models in the appendix); benchmarks are English.

References

Kirill Afendulev, Alexey Dontsov, Elena Tutubalina, and Anton Korznikov. 2026. What Attention Recalls and Recurrence Controls in Hybrid Language Models. *arXiv preprint arXiv:2609.04434*.

Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 117–134.

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Abdulrahman Alabdulkareem, Irene Tenison, Yang Zhang, Yada Zhu, Boris Katz, and Dan Gutfreund. 2026. Swarm Telepathy: Prefill-Free KV-Cache Transfer Across Families. *arXiv preprint arXiv:2610.15095*.

Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. 2025a. Relaxed Recursive Transformers: Effective Parameter Sharing with Layer-wise LoRA. In *International Conference on Learning Representations (ICLR)*. ArXiv:2410.20672.

Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoun Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. 2025b. Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2507.10524.

Andrea Banino, Jan Balaguer, and Charles Blundell. 2021. PonderNet: Learning to Ponder. *arXiv preprint arXiv:2107.05407*.

Benjamin Bergner, Andrii Skliar, Amelie Royer, Tijmen Blankevoort, Yuki Asano, and Babak Ehteshami Bejnordi. 2024. Think Big, Generate Quick: LLM-to-SLM for Fast Autoregressive Decoding. *arXiv preprint arXiv:2402.16844*.

William Brandon, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan-Kelley. 2024. Reducing Transformer Key-Value Cache Size with Cross-Layer Attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37.

Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. 2024. PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. *arXiv preprint arXiv:2406.02069*.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv preprint arXiv:2302.01318*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *arXiv preprint arXiv:1803.05457*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.

Luciano Del Corro, Allie Del Giorno, Sahaj Agarwal, Bin Yu, Ahmed Awadallah, and Subhabrata Mukherjee. 2023. SkipDecode: Autoregressive Skip Decoding with Batching and Caching for Efficient LLM Inference. *arXiv preprint arXiv:2307.02628*.

Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691*.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. 2019. Universal Transformers. In *International Conference on Learning Representations (ICLR)*. ArXiv:1807.03819.

Xin Dong, Yonggan Fu, Shizhe Diao, Wonmin Byeon, Zijia Chen, Ameya Sunil Mahabaleshwarkar, Shih-Yang Liu, Matthijs Van Keirsbilck, Min-Hung Chen, Yoshi Suhara, Yingyan Lin, Jan Kautz, and Pavlo Molchanov. 2025. Hymba: A Hybrid-head Architecture for Small Language Models. In *International Conference on Learning Representations (ICLR)*. ArXiv:2411.13676.

Zhixu Du, Yuhan Cheng, Hai Li, and Yiran Chen. 2026. Not All Loops Cost the Same: Phase-Dependent Depth for Serving Looped Transformers. *arXiv preprint arXiv:2611.07211*.

Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. 2020. Depth-Adaptive Transformer. In *International Conference on Learning Representations (ICLR)*.

Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed A. Aly, Beidi Chen, and Carole-Jean Wu. 2024. LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.

Siqi Fan, Xuezhi Fang, Xingrun Xing, Peng Han, Shuo Shang, and Yequan Wang. 2025a. Position-Aware Depth Decay Decoding (D^3): Boosting Large Language Model Inference Efficiency. In *Findings of the Association for Computational Linguistics: ACL 2025*. ArXiv:2503.08524.

Ying Fan, Yilun Du, Kannan Ramchandran, and Kangwook Lee. 2025b. Looped Transformers for Length Generalization. In *International Conference on Learning Representations (ICLR)*. ArXiv:2409.15647.

Maxim Fishman, Simeng Sun, Shantanu Acharya, Faisal Ladhak, Brian Chmiel, Santiago Akle Serano, Ron Banner, Boris Ginsburg, and Krishna C. Puvvada. 2026. Dory1.0-4B: A looped normalized-GPT. *arXiv preprint arXiv:2611.03998*.

Tianyu Fu, Zihan Min, Hanling Zhang, Jichao Yan, Guohao Dai, Wanli Ouyang, and Yu Wang. 2026. Cache-to-Cache: Direct Semantic Communication Between Large Language Models. In *International Conference on Learning Representations (ICLR)*. ArXiv:2510.03215.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. 2025. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2502.05171.

Angeliki Giannou, Shashank Rajput, Jy yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. 2023. Looped Transformers as Programmable Computers. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pages 11398–11442.

Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. 2024. Think Before You Speak: Training Language Models With Pause Tokens. In *International Conference on Learning Representations (ICLR)*.

Alex Graves. 2016. Adaptive Computation Time for Recurrent Neural Networks. *arXiv preprint arXiv:1603.08983*.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. 2025. Training Large Language Models to Reason in a Continuous Latent Space. In *Conference on Language Modeling (COLM)*. ArXiv:2412.06769.

Junhui He, Zhihui Fu, Jun Wang, and Qingan Li. 2026. POP: Prefill-Only Pruning for Efficient Large Model Inference. In *Findings of the Association for Computational Linguistics: ACL 2026*. ArXiv:2602.03295.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021a. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations (ICLR)*. ArXiv:2009.03300.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021b. Measuring Mathematical Problem Solving With the MATH Dataset. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. ArXiv:2103.03874.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models? In *Conference on Language Modeling (COLM)*. ArXiv:2404.06654.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*. ArXiv:2106.09685.

Zhiheng Hu, Yixun Wei, Jian Zhou, Yizhuang Zhou, Ji Li, Xing Chen, Yang Li, Bojun Wang, Yibo Zhu, Xiangyu Zhang, and Daxin Jiang. 2026. KITE: KV-Invariant Transformer Expansion for Efficient Agentic LLM Scaling. *arXiv preprint arXiv:2609.27294*.

Benhao Huang, Chufan Shi, Junlin Chen, Shicheng Wen, Zhengzhong Liu, Eric Xing, and Xuezhe Ma. 2026. Towards Looped Models Done Right, Part II: Rethinking at Fixed Points. *arXiv preprint arXiv:2610.06833*.

Jungo Kasai, Nikolaos Pappas, Hao Peng, James Cross, and Noah A. Smith. 2021. Deep Encoder, Shallow Decoder: Reevaluating Non-autoregressive Machine Translation. In *International Conference on Learning Representations (ICLR)*. ArXiv:2006.10369.

Woojeong Kim, Junxiong Wang, Jing Nathan Yan, Mohamed Abdelfattah, and Alexander M. Rush. 2025. OverFill: Two-Stage Models for Efficient Language Model Decoding. *arXiv preprint arXiv:2508.08446*.

Andreas Kirsch. 2026. LSTM-Style Looped Transformers with Shared KV Caches. *arXiv preprint arXiv:2610.33360*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.

Faisal Ladhak, Shantanu Acharya, Santiago Akle Serano, Simeng Sun, Boris Ginsburg, and Krishna C Puvvada. 2026. One Model to Loop Them All: Elastic Latent Recurrence for Language Models. *arXiv preprint arXiv:2610.12582*.

Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2020. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. In *International Conference on Learning Representations (ICLR)*.

John Langford, Nathan Godey, Giovanni Monea, Yoav Artzi, Harry Dong, Ying Fan, Gustavo de Rosa, and Zheng Zhan. 2026. Almost Free State Prediction Separation. *arXiv preprint arXiv:2609.03807*.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM Knows What You are Looking for Before Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's Verify Step by Step. In *International Conference on Learning Representations (ICLR)*. ArXiv:2305.20050.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2305.01210.

Liming Liu, Mingze Wang, and Tuo Zhao. 2026. Decode-Branch Transformers: Decoupling the Primary Prefill Path from Additional Decode Computation. *arXiv preprint arXiv:2608.12385*.

Mingyuan Ma, Yangmin Li, Harris Nover, Zheyu Fu, Po-Han Huang, Alex Settle, and Ashwin Nanjappa. 2026. Shallow encoding emits deep memories. *arXiv preprint arXiv:2611.06084*.

Giovanni Monea, Keshav Ramji, Yousef El-Kurdi, Luis A. Lastras, Yoav Artzi, Nathan Godey, and Ramón Fernandez Astudillo. 2026. The Surprising Effectiveness of Shared Memory in Looped Transformers. *arXiv preprint arXiv:2610.02383*.

James O' Neill and Fergal Reid. 2026. Looped Latent Attention: Cross-Loop KV Compression for Looped Transformers. *arXiv preprint arXiv:2607.15456*.

Jungsuk Oh, Hyeseo Jeon, Hyunjune Ji, Kyongmin Kong, and Jay-Yoon Lee. 2026. Shallow Prefill, Deep Decoding: Efficient Long-Context Inference via Layer-Asymmetric KV Visibility. *arXiv preprint arXiv:2605.06105*.

Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132.

Hayden Prairie, Jacob M. Ryan, Taylor Berg-Kirkpatrick, and Daniel Y. Fu. 2026. BeNice! Closing Looping's Inference-Memory Gap via KV-Sharing. *arXiv preprint arXiv:2610.31633*.

Aurick Qiao, Zhewei Yao, Samyam Rajbhandari, and Yuxiong He. 2025. SwiftKV: Fast Prefill-Optimized Inference with Knowledge-Preserving Model Transformation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. ArXiv:2410.03960.

Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST)*, pages 155–170. ArXiv:2407.00079.

Haomiao Qiu, Yining Tang, Zijia Song, Ruizi Han, Ziyue Qiao, and Miao Zhang. 2026. CacheLink: Functional KV Transport for Zero-Prefill Cross-Model Context Handoff. *arXiv preprint arXiv:2610.15509*.

David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. 2024. Mixture-of-Depths: Dynamically Allocating Compute in Transformer-Based Language Models. *arXiv preprint arXiv:2404.02258*.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2023. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. *arXiv preprint arXiv:2311.12022*.

Aishwarya P S, Pranav Ajit Nair, Yashas Samaga, Toby Boyd, Sanjiv Kumar, Prateek Jain, and Praneeth Netrapalli. 2024. Tandem Transformers for Inference Efficient LLMs. *arXiv preprint arXiv:2402.08644*.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2020. WinoGrande: An Adversarial Winograd Schema Challenge at Scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. 2025. Reasoning with Latent Thoughts: On the Power of Looped Transformers. In *International Conference on Learning Representations (ICLR)*. ArXiv:2502.17416.

Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. 2022. Confident Adaptive Language Modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Kristian Schwethelm, Daniel Rueckert, and Georgios Kaissis. 2026. Depth-adaptive Inference of Looped Language Models via Continuous Depth Batching. *arXiv preprint arXiv:2608.09444*.

Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. *arXiv preprint arXiv:1911.02150*.

Zhenyi Shen, Junru Lu, Wei Liu, Jiangnan Ye, Jiazheng Li, Di Yin, Xing Sun, Tameem Adel, Yali Du, Lin Gui, and Yulan He. 2026. On the opportunities and challenges of prefill-to-autoregressive cross-model transfer. https://github.com/zhenyi4/platour. Paper and code.

Dingjie Song, Zhiling Yan, Yuxuan Zhang, and Lichao Sun. 2026. LEV: Thinking Fast and Deep with Looped Decision Models. *arXiv preprint arXiv:2610.20951*.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2024. RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, 568:127063. ArXiv:2104.09864.

Yutao Sun, Li Dong, Tianzhu Ye, Shaohan Huang, Jianyong Wang, and Furu Wei. 2026. Universal YOCO for Efficient Depth Scaling. *arXiv preprint arXiv:2604.01220*.

Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. 2024. You Only Cache Once: Decoder-Decoder Architectures for Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2405.05254.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. 2023. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. In *Findings of the Association for Computational Linguistics: ACL 2023*. ArXiv:2210.09261.

Victor Conchello Vendrell, Arnau Padres Masdemont, Niccolò Grillo, Jordi Ros-Giralt, Arash Behboodi, and Fabio Valerio Massoli. 2026. Memory-Efficient Looped Transformer: Decoupling Compute from Memory in Looped Language Models. *arXiv preprint arXiv:2605.07721*.

Simon P. Villani. 2026. LatentPort: Beyond KV Cache - Cross-Model Transfer of Recurrent Memory in Hybrid Language Models: A 4B-to-9B Hybrid-State Handoff Without Target Prefix Replay. *arXiv preprint arXiv:2609.25053*.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. 2024. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. ArXiv:2406.01574.

Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, Siyuan Qiao, and Xingyan Bin. 2025a. Parallel Loop Transformer for Efficient Test-Time Computation Scaling. *arXiv preprint arXiv:2510.24824*.

Bohong Wu, Shen Yan, Sijun Zhang, Jianqiao Lu, Yutao Zeng, Ya Wang, and Xun Zhou. 2025b. Efficient Pretraining Length Scaling. *arXiv preprint arXiv:2504.14992*.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations (ICLR)*.

Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. 2020. DeeBERT: Dynamic Early Exiting for Accelerating BERT Inference. In *Proceed-*

ings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL).

Jian Yang, Shawn Guo, Wei Zhang, Tianyu Zheng, Yaxin Du, Haau-Sing Li, Jiajun Wu, Yue Song, Yan Xing, Qingsong Cai, Zelong Huang, Chuan Hao, Ran Tao, Xianglong Liu, Wayne Xin Zhao, Mingjie Tang, Weifeng Lv, Ming Zhou, and Bryan Dai. 2026. LoopCoder-v2: Only Loop Once for Efficient Test-Time Computation Scaling. *arXiv preprint arXiv:2606.18023*.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2025. Gated Delta Networks: Improving Mamba2 with Delta Rule. In *International Conference on Learning Representations (ICLR)*. ArXiv:2412.06464.

Wanqi Yang and Shiwei Liu. 2026. FlashLoop: Fast and memory-efficient looped transformers via lazy updates. *arXiv preprint arXiv:2609.29812*.

Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. 2025. HELMET: How to Evaluate Long-Context Language Models Effectively and Thoroughly. In *International Conference on Learning Representations (ICLR)*. ArXiv:2410.02694.

Vincent-Daniel Yun, Woosang Lim, Haneul Yoo, Sungjoo Yoo, Murali Annavaram, and Sai Praneeth Karimireddy. 2026. Prefill-Free Cross-Family KV Cache Transfer for Heterogeneous Multi-Agent LLMs. *arXiv preprint arXiv:2609.32259*.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a Machine Really Finish Your Sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*.

Jiamu Zhang and Hanjie Chen. 2026. BellowsKV: One Latent for Many Layers, Expanded on Demand. *arXiv preprint arXiv:2610.35023*.

Yudong Zhang, Wei Ding, Kaili Zheng, Yuchen Ding, and Zhenyu Zhang. 2026. Accordion: Elastic-Depth Inference via Fixed-Prefix Layer Looping. *arXiv preprint arXiv:2610.32492*.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H_2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.

Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. Instruction-Following Evaluation for Large Language Models. *arXiv preprint arXiv:2311.07911*.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, and 14 others. 2025. Scaling Latent Reasoning via Looped Language Models. *arXiv preprint arXiv:2510.25741*.

Zayd Muhammad Kawakibi Zuhri, Muhammad Farid Adilazuarda, Ayu Purwarianti, and Alham Fikri Aji. 2025. MLKV: Multi-Layer Key-Value Heads for Memory Efficient Transformer Decoding. In *Findings of the Association for Computational Linguistics: NAACL 2025*.

Guide to the appendix. Appendices A–C, E–H and R support the main text; Appendix D gives the per-benchmark results behind Table 1 together with earlier per-benchmark studies. Appendices I–Q report earlier studies, mostly with the one-cache variant of the loop adapter (decode loops read only the last prefill loop's cache, Section 6), and the other checkpoints; Appendix S lists what is released. In the appendix, "native n/n" is Original n/n, "LoRA-distilled" is Distilled, and "loop adapter" or "aligned adapter" is Ours.

A Per-Request Time

Table 2 gives the per-request distributions behind Figure 1: medians and 90th percentiles of the output length, the time per request and the prefill share, over the same 100 requests per task for every configuration (batch 1, one B300, greedy). At the 90th percentile the prefill share stays below 5% on GSM8K, MATH-500 and BBH; on MMLU-Pro, where many answers are short, it reaches 5–17%. The time per request follows the output length, whose 90th percentile is up to about four times its median within a task.

Every time is the mean of two passes over the same requests in opposite orders on a machine running nothing else; the answers are identical in both. Between the passes the mean time per request of a configuration and task differs by 5.9% at the median and by up to 22.5% (128 configuration-task pairs). An earlier pass, on a machine that also ran evaluations, is not used. Folding the maps changes floating-point rounding, so greedy answers can differ: on the four tasks Ours 4/1 scores 41.2 with folded maps and 43.0 with per-step maps.

task	configuration	input	output	output p90	s / request	p90	prefill %	p90
GSM8K	Ours 1/1	318	88	142	4.1	6.6	2.38	3.55
GSM8K	Ours 1/2	318	96	143	9.3	13.7	1.64	2.32
GSM8K	Original 1/1	318	90	171	3.2	6.0	2.65	4.50
GSM8K	Original 2/2	318	94	157	6.5	10.8	2.23	3.20
GSM8K	Original 4/4	318	94	145	13.1	20.3	2.09	3.14
MATH-500	Ours 1/1	743	601	1024	27.1	45.9	0.40	0.81
MATH-500	Ours 1/2	743	491	1024	48.9	101.8	0.32	0.61
MATH-500	Original 1/1	743	631	1024	25.9	42.0	0.36	0.81
MATH-500	Original 2/2	743	324	1024	22.0	70.1	0.64	1.56
MATH-500	Original 4/4	743	260	1024	36.3	139.9	0.78	1.83
BBH	Ours 1/1	984	168	332	7.9	16.1	1.46	2.84
BBH	Ours 1/2	984	164	333	14.7	30.4	1.07	2.03
BBH	Original 1/1	984	168	1024	5.9	35.6	1.56	3.77
BBH	Original 2/2	984	168	344	12.7	26.0	1.32	2.77
BBH	Original 4/4	984	156	343	21.6	47.2	1.28	2.33
MMLU-Pro	Ours 1/1	1331	57	170	2.7	8.0	4.52	7.43
MMLU-Pro	Ours 1/2	1331	52	214	4.9	20.1	3.23	5.45
MMLU-Pro	Original 1/1	1331	44	143	1.6	5.0	6.70	17.15
MMLU-Pro	Original 2/2	1331	60	157	4.2	10.9	3.84	6.56
MMLU-Pro	Original 4/4	1331	60	248	8.3	34.6	3.55	5.80

Table 2: **Per-request time, Ouro-2.6B.** Medians unless marked p90 (90th percentile); input and output in tokens; prefill %: prefill time over total time per request.

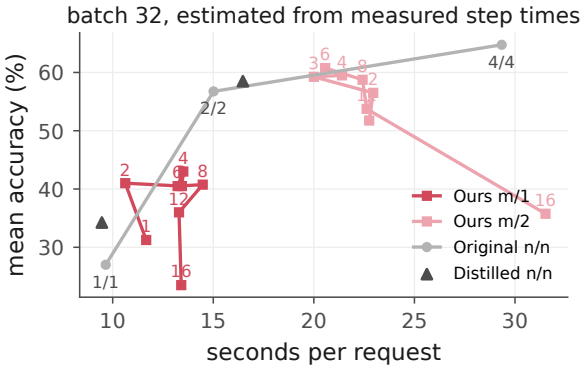


Figure 4: **Figure 3 at batch 32**, estimated, with per-step maps for Ours: each request's measured prefill time plus its measured output tokens times the decode step time measured at batch 32 for its prompt length (512 or 1,536 tokens). An estimate, not a serving measurement.

B Where the Gain Stops

For each benchmark and decode depth, the stopping depth m^* is the first sampled prefill depth that none of the next three sampled depths beats by more than 0.5 points on a development half of the items (split by a hash of the item id); the accuracy at m^* and at the deepest evaluated depth is reported on the other half (Table 3). The rule has no interval or equivalence test. Settings where no evaluated depth meets it, because accuracy still rises or too few deeper depths were evaluated, are left out: for the main adapter MMLU, MMLU-Pro, BBH and ARC-C at $n = 1$, MMLU-Pro and ARC-C at $n = 2$, and HellaSwag and Winogrande at both; for the adapter trained up to 16, MMLU-Pro and ARC-C at $n = 2$. Over three training seeds of that adapter, 16 settings have a stop in every seed; the three stops are equal in 8 of them, within one sampled depth in 11 and all at two to four loops in 9, and MATH-500 stops at one prefill loop in every seed.

C Prefill Depth on Ouro-1.4B

Table 4 repeats the sweep of Section 4 on Ouro-1.4B with its loop adapter (trained on $m, n \leq 4$), on the same benchmarks and protocol.

D Extended Results

Setup. Models: Ouro-2.6B, Ouro-1.4B and their Thinking variants (Zhu et al., 2025) (four loops) and SenseNova-Loop-9B-Preview, a hybrid of Gated-DeltaNet and full-attention layers whose middle block is visited twice. Benchmarks cover knowledge and commonsense (MMLU (Hendrycks et al., 2021a), MMLU-Pro (Wang et al., 2024), ARC-Challenge (Clark et al., 2018), GPQA-Diamond (Rein et al., 2023), HellaSwag (Zellers et al., 2019), Winogrande (Sakaguchi et al., 2020)), long-context retrieval (RULER (Hsieh et al., 2024) at 4K/8K/16K, HELMET (Yen et al., 2025)), reasoning and instruction following (BBH (Suzgun et al., 2023), IFEval (Zhou et al., 2023)), math (GSM8K (Cobbe et al., 2021), MATH-

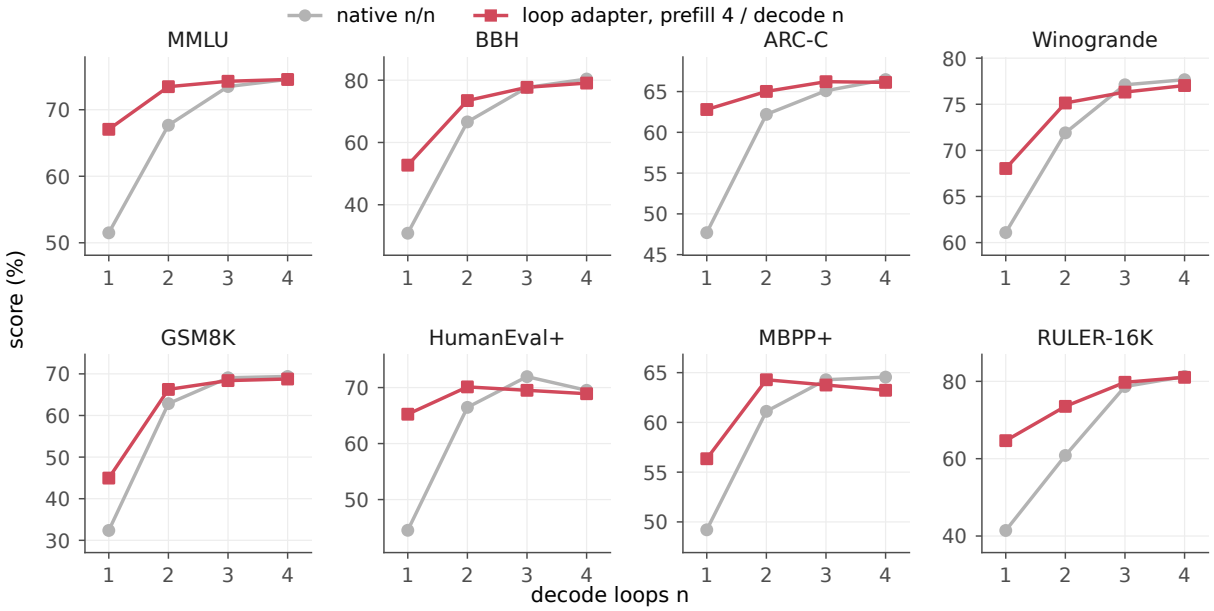


Figure 5: **Decode cheaply, prefill deeply.** Ouro-2.6B, accuracy (%) against the number of decode loops n. Grey: the native model, which runs n loops for both prefill and decode (n/n). Red: one loop adapter (aligned caches) that prefills with four loops and decodes with n. With few decode loops, four prefill loops recover much of the accuracy of the deeper native model; with three or four it matches native. Each panel uses the items all points share; all settings and every (m, n) cell: Appendix K and Table 11.

500 (Hendrycks et al., 2021b; Lightman et al., 2024)) and code (HumanEval+ and MBPP+ (Chen et al., 2021; Liu et al., 2023)). We follow the base-model protocol of the Ouro models where it is available (shots, chain of thought, strict-match scoring for GSM8K, final-answer scoring for MATH-500, evalplus for code), with documented substitutions; protocols, set sizes and the effect of departing from them are in Appendix H. One loop adapter per model is trained once; on Ouro-2.6B and Ouro-1.4B it is evaluated at all 16 pairs, on the Thinking models at the pairs reported, and on SenseNova-Loop-9B at prefill 1 and 2 with one decode visit. We compare with the native model at the same number of decode loops. Every comparison is paired over items with 95% bootstrap intervals.

D.1 The task map

Figure 5 plots accuracy against decode loops. The native model loses most of its accuracy when it decodes with fewer loops; the loop adapter, prefilling with four loops, keeps much of it: at one decode loop MMLU rises from 51.5 to 67.1, BBH from 30.9 to 52.7 and HumanEval+ from 44.5 to 65.2. Decode loops matter most for multi-step math (native GSM8K: 32.4 at one loop, 69.4 at four), and the adapter adds 13 points there at one decode loop. With three or four decode loops the adapter tracks the native model, also on long-context retrieval (RULER-16K at 4/3: 79.8 against 78.7 for Original 3/3).

D.2 Main results

Table 5 compares fixed configurations at the same number of decode loops. With one decode loop, prefilling with four loops raises the average from 44.2 (native 1/1) to 56.8, above native 1/1 distilled on the same data (49.8); with two, the aligned adapter averages 67.2, against 63.0 for native 2/2 and 65.3 for its distilled version, within 3.0 points of native 4/4 with half the decode loops. The one-cache variant, at a fraction of the prompt memory, gives up under one point (56.4 and 66.4). The adapter is what makes untying work: without it, decoding from the last prefill loop's KV with Ouro's own reuse of the last loop's KV, the untied configurations average 26.6 and 30.9, below even native 1/1 (on GSM8K at 4/2, 11.7 against 66.7 with the adapter). Within the same adapter, deeper prefill alone adds up to 15 points (Table 10), so the gain is not only extra training. Choosing the prefill count per benchmark on held-out items instead of fixing it at four beats native 1/1 significantly on 14 of 15 settings (Appendix J), and Ouro-1.4B shows the same pattern (Appendix L).

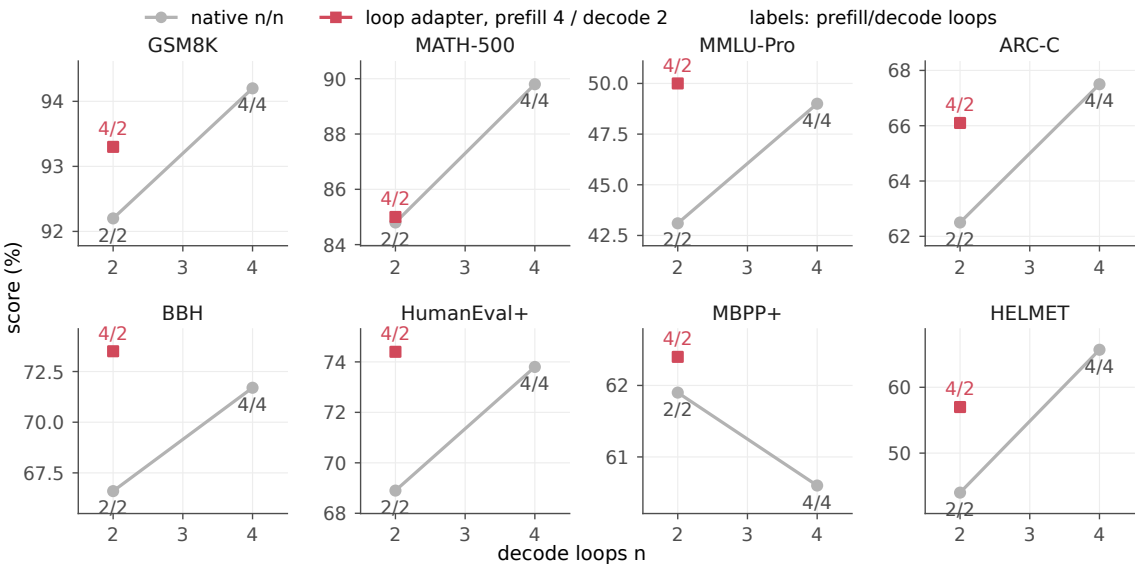


Figure 6: **Prefill 4 / decode 2: two decode loops, point estimates within two points of four on six of eight benchmarks.** Ouro-2.6B-Thinking, score against the number of decode loops. Grey: native 2/2 and 4/4. Red: the loop adapter (one cache) with prefill 4 / decode 2. Same items for all three in each panel; HELMET is averaged over its categories. Table 16 gives every score, the paired intervals and the same adapter at prefill 2 / decode 2.

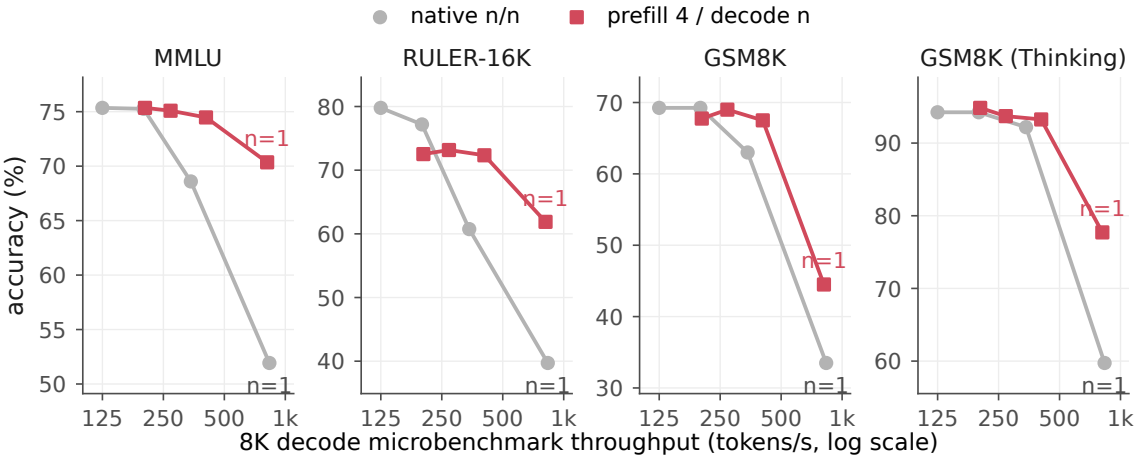


Figure 7: **At one or two decode loops, the adapter is more accurate than native n/n.** Native n/n (grey) and the loop adapter with prefill 4 / decode n (red, one cache); each line runs from $n = 1$ (labelled, right) to $n = 4$ (left; Thinking adapter $n = 1$–3). Throughput is a separate decode microbenchmark: best decode tokens/s over batch sizes at 8K context on one B300 (same for the base and Thinking models, which share the architecture); one stored prompt KV cache admits larger batches as n grows. Ouro-2.6B on MMLU (20 questions per subject), RULER-16K (260 items) and GSM8K (400 problems); Ouro-2.6B-Thinking on all 1,319 GSM8K problems.

Equal decode loops do not mean equal latency (Section D.4).

Training cost. The adapter needs little data (Table 6): trained on 6M answer tokens, an eighth of the default budget, it already averages 53.7 and 65.2 at prefill 4 / decode 1 and 2, against 56.4 and 66.4 with 48M, and neither a smaller nor a larger LoRA rank changes the result.

D.3 More prefill loops, fewer decode loops

Figure 6 applies the finding to a reasoning model, Ouro-2.6B-Thinking, at prefill 4 / decode 2. With the same number of decode loops as Original 2/2, Ours 4/2 scores higher on all eight benchmarks, by up to 13 points on HELMET and 7 on MMLU-Pro and BBH; against Original 4/4, which decodes with twice the loops, it is within two points on six of the eight. The prefill loops themselves carry the gain: the same adapter at Ours 2/2 is up to 8 points

adapter	benchmark	n	m^*	test m=1	tes
trained ≤ 4	GSM8K	1	2	36.5	
trained ≤ 4	MATH-500 4-shot	1	1	36.2	
trained ≤ 4	HumanEval+	1	3	43.2	
trained ≤ 4	MBPP+	1	4	48.0	
trained ≤ 4	RULER-16K	1	4	52.2	
trained ≤ 4	MMLU	2	4	52.7	
trained ≤ 4	BBH 3-shot	2	3	55.4	
trained ≤ 4	GSM8K	2	3	56.4	
trained ≤ 4	MATH-500 4-shot	2	1	66.2	
trained ≤ 4	HumanEval+	2	3	59.3	
trained ≤ 4	MBPP+	2	5	55.6	
trained ≤ 4	RULER-16K	2	3	57.3	
trained ≤ 16	MMLU	1	3	50.7	
trained ≤ 16	MMLU-Pro	1	5	28.0	
trained ≤ 16	BBH 3-shot	1	3	36.9	
trained ≤ 16	ARC-C	1	3	50.1	
trained ≤ 16	GSM8K	1	2	39.0	
trained ≤ 16	MATH-500 4-shot	1	1	36.6	
trained ≤ 16	HumanEval+	1	5	48.1	
trained ≤ 16	MBPP+	1	2	48.0	
trained ≤ 16	RULER-16K	1	2	52.8	
trained ≤ 16	MMLU	2	3	52.9	
trained ≤ 16	BBH 3-shot	2	3	58.1	
trained ≤ 16	GSM8K	2	3	57.3	
trained ≤ 16	MATH-500 4-shot	2	1	65.8	
trained ≤ 16	HumanEval+	2	3	61.7	
trained ≤ 16	MBPP+	2	4	57.1	
trained ≤ 16	RULER-16K	2	3	58.9	

Table 3: **Stopping depth of the prefill gain.** "trained $\leq$ 4": the main adapter (beyond four it extrapolates); "trained $\leq$ 16": the adapter trained on $m \in \{1, 2, 3, 4, 6, 8, 12, 16\}$. Deepest: the deepest evaluated depth, in parentheses.

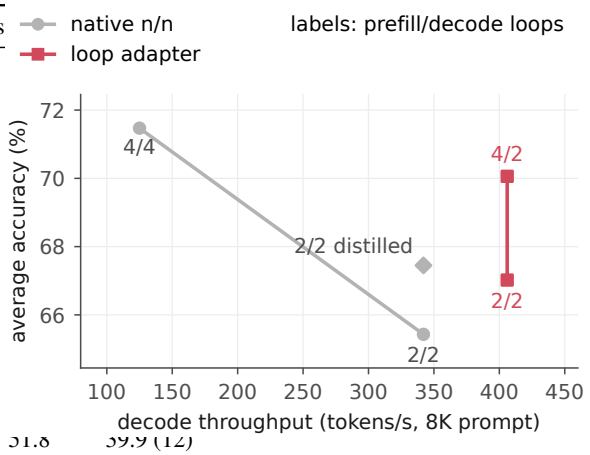


Figure 8: **Prefill 4 / decode 2 comes within 1.4 points of native 4/4 at 3.2× its decode throughput.** Ouro-2.6B-Thinking average accuracy over the eight benchmarks of Figure 6 (each on the items all five configurations share) against decode throughput (best tokens/s over batch sizes at an 8K-token prompt, Table 19). Labels: prefill/decode loops. The adapter decodes at the same cost as 2/2 and 4/2, and distilled native 2/2 (LoRA merged) at that of native 2/2; the four prefill loops add 3.1 points. HELMET is averaged per dataset, then per category.

lower on MMLU-Pro and HELMET, and on Ouro-1.4B-Thinking four prefill loops instead of two are better on six of eight benchmarks (Appendix M). Ours 4/2 also scores above native 2/2 distilled on the same data on all eight (Table 17).

Against measured decode throughput (Figure 7), prefill 4 / decode n is more accurate than Original n/n at one or two decode loops and has higher throughput at three or four, because the one stored prompt cache admits larger batches; prefill 4 / decode 2 runs at 1.19× the throughput of Original 2/2 and 3.2× that of Original 4/4. The gain grows with the prompt: against Original 4/4 it is 2.4× at a 2K-token prompt and 3.3× at 15K, where Ours 4/2 also outpaces Original 2/2 (1.12×). Aligned caches decode at about the cost of Original n/n (314 against 342 tokens/s at $n = 2$, 2.5× Original 4/4).

D.4 End-to-end time

We time whole answers at batch 1, prefill included (Table 18 in Appendix N). On Ouro-2.6B-Thinking, prefill 4 / decode 2 answers 192 timed GSM8K problems in 0.59× [0.52, 0.66] the time of Original 4/4. On all 1,319 problems it scores 93.3 against 94.2, a paired difference of −1.0 [−2.0, 0.1]. Prefilling with 4 rather than 2 loops before decoding with two changes the answer time by 1.02× [0.92, 1.13], and prefill 4 / decode 2 takes 1.20× [1.05, 1.38] the time of the distilled native 2/2, at similar accuracy on these problems: the saving comes from decoding with two loops, and what the extra prefill loops add is the accuracy of Section D.3. On MATH-500 the saving is smaller: 0.63× the time of Original 4/4 but 1.22× that of Original 2/2 (Appendix N). A smaller model is not the same trade: native Ouro-1.4B-Thinking 4/4 takes about as long per answer (65.2 against 69.6 seconds), and Ouro-2.6B-Thinking at prefill 4 / decode 2 scores higher on five of the eight benchmarks of Figure 6, by 3.5 to 3.8 points on MMLU-Pro, ARC-Challenge and BBH. On SenseNova-Loop-9B the trained Ours 1/1 scores 95.6 against 95.8 for Original 2/2 on all 1,319 GSM8K problems and takes 0.59× [0.52, 0.68] the time per answer, close to the ratio of layer executions per token (32 vs. 56).

D.5 Looped decision models: both directions

Untying is not specific to generation. Lev (Song et al., 2026) turns looped language models into de-

benchmark	n	m=1	2	3	4	5	6	8	12	16
MMLU	1	42.8	56.9	**63.7**	63.6	60.2	59.0	56.1	46.1	31.9
MMLU-Pro	1	20.0	30.0	36.4	**36.7**	29.1	30.0	28.1	20.2	12.1
BBH 3-shot	1	**33.1**	9.5	9.3	13.4	9.3	8.6	14.1	18.5	7.5
ARC-C	1	45.4	52.2	54.9	**56.3**	52.7	53.8	52.7	45.6	35.6
GSM8K	1	21.3	27.2	**29.6**	25.7	15.2	17.6	21.2	18.3	9.9
MATH-500 4-shot	1	**28.0**	24.0	27.6	24.2	15.2	21.8	15.2	10.6	9.8
HumanEval+	1	40.2	39.6	**46.3**	39.6	29.9	37.2	34.1	28.1	0.0
MBPP+	1	42.3	47.4	**49.2**	47.6	38.9	43.9	43.9	33.1	0.0
RULER-16K	1	43.4	45.9	45.1	**47.2**	36.8	43.1	43.5	40.0	29.2
MMLU	2	43.6	60.6	66.2	**66.7**	65.2	62.6	60.1	50.2	35.6
MMLU-Pro	2	24.3	36.9	44.3	43.8	44.3	**45.5**	38.8	34.3	23.6
BBH 3-shot	2	46.2	52.7	**53.1**	49.4	47.5	32.6	39.8	36.2	16.6
ARC-C	2	51.7	56.2	58.2	**58.7**	58.1	57.7	56.3	50.3	39.6
GSM8K	2	41.1	46.6	**53.3**	51.1	40.8	39.7	43.0	39.5	23.6
MATH-500 4-shot	2	53.8	**57.4**	50.8	53.2	43.8	46.0	36.6	23.2	22.0
HumanEval+	2	61.0	56.1	**63.4**	62.8	54.3	51.2	50.0	47.0	0.0
MBPP+	2	53.2	55.8	**57.7**	57.1	49.2	50.3	49.5	42.6	0.0
RULER-16K	2	50.6	**59.6**	49.9	53.5	45.7	47.8	50.5	42.1	35.3

Table 4: **Prefill depth on Ouro-1.4B.** Accuracy (%) against prefill loops m at decode n; best m of each row in bold; –: not evaluated.

n	method	MMLU	MMLU-Pro	BBH	ARC-C	HellaSw.	Winogr.	GSM8K	MATH	HE+	MBPP+	RULER-16K	Avg.
1	Original 1/1	51.5	26.9	30.9	47.7	69.0	61.1	32.4	32.2	44.5	49.2	41.4	44.2
1	Training-free 4/1	61.9	0.0	0.8	37.5	40.4	51.9	8.9	4.6	29.3	48.9	8.0	26.6
1	Distilled 1/1	52.7	26.7	40.4	55.0	70.1	66.5	40.4	**37.0**	54.3	50.3	54.7	49.8
1	Distilled 1/1, rank 80	53.4	27.9	40.5	55.3	70.2	66.6	40.3	36.4	55.5	50.0	56.2	50.2
1	Prefill-4 LoRA 4/1	**68.1**	**47.4**	**54.4**	62.9	**72.7**	67.5	47.2	25.0	**67.7**	**57.4**	58.3	**57.1**
1	Ours 4/1, maps frozen	66.8	46.9	49.9	**63.0**	72.5	65.9	**47.5**	23.4	64.0	55.3	49.8	55.0
1	Ours, one cache 4/1	66.6	**47.4**	51.7	62.5	72.4	67.1	44.7	25.8	64.6	55.3	62.8	56.4
1	**Ours, aligned 4/1**	67.1	45.0	52.7	62.8	72.7	**68.0**	45.0	25.4	65.2	56.4	**64.7**	56.8
2	Original 2/2	67.7	45.0	66.6	62.2	77.8	71.9	62.9	**50.6**	66.5	61.1	60.8	63.0
2	Training-free 4/2	71.3	4.0	4.0	41.0	50.6	65.5	11.7	0.0	7.3	33.1	52.0	30.9
2	Distilled 2/2	67.2	46.2	70.5	62.9	77.6	74.6	69.6	46.8	69.5	63.0	70.3	65.3
2	Distilled 2/2, rank 80	66.9	44.8	70.5	62.6	77.4	75.2	**70.1**	48.2	69.5	62.7	71.8	65.4
2	Prefill-4 LoRA 4/2	73.2	**54.0**	72.8	**65.4**	78.2	**75.3**	67.7	47.2	68.9	63.0	**74.5**	**67.3**
2	Ours 4/2, maps frozen	73.2	52.6	73.0	64.8	**78.3**	74.2	67.2	47.8	69.5	62.4	71.6	66.8
2	Ours, one cache 4/2	73.3	51.2	71.2	65.1	78.0	75.1	66.7	47.0	68.3	62.7	72.2	66.4
2	**Ours, aligned 4/2**	**73.5**	**54.0**	**73.5**	65.0	78.2	75.1	66.3	45.8	**70.1**	**64.3**	73.5	67.2
4	Original 4/4	74.5	55.2	80.4	66.5	79.5	77.7	69.4	53.6	69.5	64.5	81.3	70.2

Table 5: **Same decode loops, different prefill.** Ouro-2.6B, accuracy (%) grouped by decode loops n. Original: the released model. Training-free: the frozen model decodes from a four-loop prefill without an adapter. Distilled: LoRA on the n-loop model distilled from native 4/4 with the adapter's data, teacher and tokens. Ours: one cache (all decode loops read prefill loop m) or aligned (decode loop k reads prefill loop $\max(m - n + k, 1)$). Each column on the items all rows share. Over three training seeds (mean $\pm$ s.d.), Ours (aligned) averages 56.7 ± 0.1 at 4/1 and 67.1 ± 0.2 at 4/2, Distilled 49.7 ± 0.3 at 1/1 and 65.1 ± 0.2 at 2/2 (rank 80: 50.2 over two seeds and 65.3 ± 0.1), the prefill-4 LoRA 56.8 ± 0.3 and 67.3 ± 0.4, and Ours with frozen maps 54.3 ± 0.6 and 66.6 ± 0.4 (its RULER-16K at 4/1: 49.8, 42.4 and 41.8 against 64.7); the one-cache adapter's seeds: Appendix I.

cision models: they read a long context, then score the options of a question with a learned head after every loop, without generating text. We prefill the context with m loops, let each of the n decision loops attend to the last prefill loop's KV, and continue Lev's own LoRA and scoring head, once for every m; the control continues the same weights on the same data for the same number of updates with Lev's native n/n forward (Appendix P).

In these runs the sign of $m - n$ gives the direction of the difference (Figure 9, Table 21). With more prefill than decision loops the model is significantly above the control on 12 of 13 (m, n) pairs, most with one decision loop, where prefilling with all of the backbone's loops gains $+5.3$ [3.4, 7.2] on Lev-1.4B, $+7.5$ [5.7, 9.5] on Lev-2.6B and $+4.4$ [2.8, 6.1] on Lev-3B. With fewer prefill than decision loops it is below the control on all 13 pairs (significantly on 11), by up to 13.0 points (-13.0 [-14.7, -11.3]; Lev-2.6B, prefill 1, four deci-

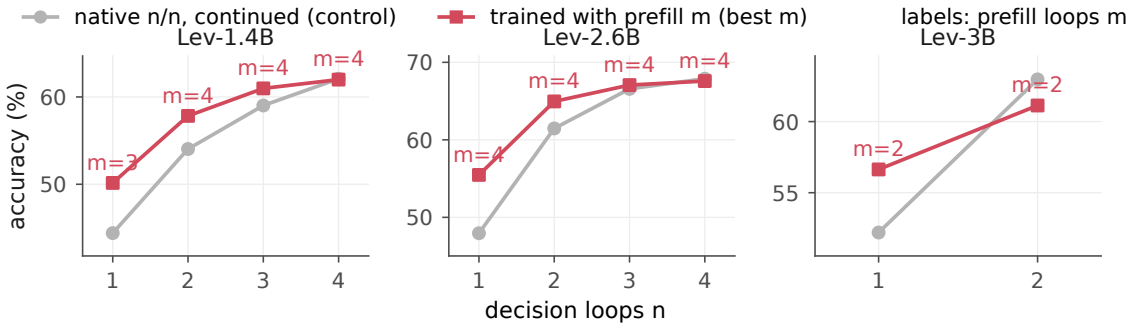


Figure 9: **Decision models gain when the context gets more loops than the decision.** Lev decision models (Song et al., 2026), LevBench development set (3,270 questions), accuracy against decision loops n. Grey: native n/n continued with the same data and updates. Red: the best of the models trained to prefill with m loops (labels); seed 0. All pairs: Table 21.

training tokens	LoRA rank	4/1 avg.	4/2 avg.
6M	16	53.7	65.2
12M	16	55.5	65.9
24M	16	55.7	66.1
48M (default)	16	56.4	66.4
12M	4	55.4	65.8
12M	64	55.3	66.5

Table 6: **The loop adapter is cheap to train.** Ouro-2.6B, average over the benchmarks of Table 5 (each on the items all runs share) at prefill 4 / decode 1 and 2, for adapters trained on fewer answer tokens of the same mixture and with other LoRA ranks.

sion loops). The fewer-prefill direction replicates over four training seeds: every seed is below the control on all 13 pairs, and on 10 of them significantly in every seed; the more-prefill comparison at the backbone's full loop count is replicated over four seeds and against a control trained twice as long (Appendix P). As in the task map, the context gains from many loops and the decision can then stop early.

E Method Details

We describe the loop adapter for Ouro; SenseNova-Loop-9B uses a lighter variant without maps (Appendix G).

Prefill and decode. "Ours m/n" prefills the prompt with m loops and decodes every answer token with n loops; Original n/n is the unmodified model. The frozen model prefills the prompt with m loops and keeps the prompt keys and values its decode loops read: with aligned caches (Ours), decode loop k reads prefill loop $\max(m-n+k, 1)$; in the one-cache variant every decode loop reads the last prefill loop. The decoder is the same frozen model with rank-16 LoRA (Hu et al., 2022) on its attention projections, running n loops per token.

RoPE-commuting maps. Decode loop k reads the prompt KV cache through a map per (loop, layer, KV head). The key map is one complex scale c_j per rotary frequency pair j (Su et al., 2024): viewing the pair of key coordinates (k_{2j}, k_{2j+1}) as a 2-D vector, $c_j = r\,e^{i\phi}$ stretches it by r and turns it by ϕ. Writing a key pair rotated for position s as a complex number $\tilde{k}_j = e^{is\theta_j}k_j$, scaling commutes with the rotation, $c_j\tilde{k}_j = e^{is\theta_j}(c_j k_j)$, and $\mathrm{Re}\langle q_j, c_j\tilde{k}_j\rangle = \mathrm{Re}\langle \bar{c}_j q_j, \tilde{k}_j\rangle$: the map is applied to the query and, in this per-step form, the stored cache is never rewritten (the timed form of Section 6 instead folds both maps into the cache once at prefill). The value map is a full linear map W_k per head; because attention is linear in the values, $\sum_i a_i W_k v_i = W_k \sum_i a_i v_i$ is applied once to the attention output over the prompt. Prompt and generated tokens share one softmax: the two parts are computed by two attention calls with outputs o_P, o_A and log-sum-exps λ_P, λ_A of their attention logits, and merged as $g\,o_P + (1-g)\,o_A$ with $g = \sigma(\lambda_P - \lambda_A)$, which equals one softmax over both (the blockwise softmax of FlashAttention (Dao, 2023)). In the one-cache variant the prompt KV is therefore stored once for every n, and total KV is $b(\ell + n\,t)$ instead of $b\,n(\ell+t)$. Serving from one cache equals materialising a mapped prompt cache per loop; in bfloat16 all 512 checked greedy tokens agree.

Aligned caches (Ours). The main method keeps one prompt cache per decode loop and lets decode loop k read prefill loop $\max(m-n+k, 1)$ through

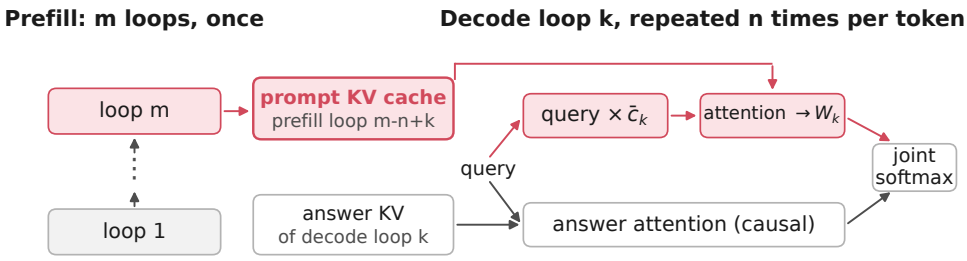


Figure 10: **The loop adapter.** The frozen model prefills the prompt with m loops. Each of the n decode loops reads the prompt KV of a prefill loop through its own map: a scale on each rotary pair of the keys (moved onto the query, so in this per-step form the cache is never rewritten) and a value map applied after attention; prompt and answer attention share one softmax. Aligned: decode loop k reads prefill loop $\max(m-n+k, 1)$; one cache: every decode loop reads loop m.

```
# blocks: frozen looped block, L layers; m/n: prefill/decode loops
# c[k][l] = 1+a+1j*b per RoPE pair; W[k][l] = I+delta (init: I)
# LoRA on q, k, v, o is active only while decoding.

def prefill(prompt, m): # prompt minus its last token
    h = embed(prompt)
    for j in range(m):
        for l in range(L):
            h, K[j][l], V[j][l] = blocks[l](h) # every loop's K, V
    return K, V

def attend(q, K, V, kv_answer, c, W): # layer l, decode loop k
    qc = complex_mul(q, c.conj()) # key scale moved to the query
    o_p, lse_p = flash_attn(qc, K, V) # prompt part
    o_p = o_p @ W.T # attention is linear in V
    o_a, lse_a = flash_attn(q, *kv_answer, causal=True)
    g = torch.sigmoid(lse_p - lse_a) # exact softmax over both
    return g * o_p + (1 - g) * o_a

def decode(token, K, V, m, n): # last prompt token, then answer
    h = embed(token)
    for k in range(n):
        j = max(m - n + k, 0) # aligned; one cache: j = m - 1
        for l in range(L):
            h = blocks[l](h, attn=lambda q, kv: attend(
                    q, K[j][l], V[j][l], kv, c[k][l], W[k][l]))
    return lm_head(h)
```

```
# training: one (m, n) per micro-batch,
# teacher = the native 4/4 model
m, n = torch.randint(1, 5, (2,)).tolist()
with torch.no_grad():
    t = native(x, loops=4) # frozen, no adapter
s = model(x, prefill=m, decode=n) # LoRA + maps
kl = kl_div(s.log_softmax(-1),
            t.log_softmax(-1), log_target=True,
            reduction="none").sum(-1)
loss = kl[answer].sum() / (answer.sum() - len(x))
```

Figure 11: **The loop adapter in PyTorch-style pseudocode.** Decode loop k reads prefill loop $m-n+k$ (loops counted from zero here; the one-cache variant keeps only the last prefill loop) through its own key scale (applied to the query) and value map (applied to the attention output), and the prompt and answer attention are combined exactly through their log-sum-exp.

its own map, so at $m = n$ every decode loop reads exactly the KV the original model would. It stores the prompt n times, as Original n/n does. The one-cache variant, trained identically, stores it once and is slightly less accurate, most of all on long-context retrieval.

What the maps do. For the aligned adapter, the same training with the maps frozen at the identity moves ten of the 11 benchmarks by at most 2.8 points at 4/1 but lowers RULER-16K from 64.7 to 49.8 (Table 5). The one-cache variant shows the same pattern: with frozen maps it is within 0.4 points on MMLU (significantly lower at 1/1 and 2/2, by 0.4 and 0.3), has no significantly negative GSM8K difference (at 4/1 it scores 2.3 points higher [0.2, 4.5]), but loses 11.4 points on RULER-16K at 4/1 (Appendix O). The maps matter for long prompts.

Baseline learning rate. The distilled controls use the adapter's learning-rate schedule. Scaling the whole schedule by 0.3 or 3 gives 11-benchmark averages of 49.2 and 50.1 for Distilled 1/1 (49.8 at the default) and 64.7 and 65.2 for Distilled 2/2 (65.3); at rank 80 with the threefold rate they are 50.0 and 65.6 (50.2 and 65.4 at the default). None reaches Ours at 4/1 or 4/2 (56.8, 67.2). The adapter trained the same way at 0.3 and 3 times its learning rate averages 55.5 and 57.4 at

4/1 and 66.8 and 67.8 at 4/2: even its weakest setting is above the best distilled control, and both methods are reported at the default rate.

Training. We distil (Hinton et al., 2015): we minimise $\mathrm{KL}(\text{teacher} \,\|\, \text{student})$ over answer positions, with the native 4/4 model as teacher, on 48M answer tokens of reasoning traces, instruction data, long-context data and web text. One (m, n) pair is sampled per micro-batch, so one set of parameters (63M, 2.4% of Ouro-2.6B) serves all 16 pairs. Details are in Appendix G.

F Extended Related Work

Looped and elastic depth. Weight-tied depth recurrence goes back to Universal Transformers and cross-layer sharing (Dehghani et al., 2019; Lan et al., 2020); looped transformers are computationally universal (Giannou et al., 2023), can simulate chain-of-thought steps in latent space (Saunshi et al., 2025) and generalise in length with adaptive step counts (Fan et al., 2025b); continuous latent reasoning (Hao et al., 2025) instead feeds hidden states back as input tokens. Relaxed Recursive Transformers (Bae et al., 2025a) convert pretrained models into looped ones with per-loop LoRA; Mixture-of-Recursions (Bae et al., 2025b) and Huginn (Geiping et al., 2025) route or scale the loop count. Accordion (Zhang et al., 2026) and Elastic Latent Recurrence (Ladhak et al., 2026) train one model for several depth budgets; ELR finds that keeping only the last loop's KV retains most of the gain, as Dory1.0-4B (Fishman et al., 2026) does at any loop count. The Parallel Loop Transformer (Wu et al., 2025a), LoopCoder-v2 (Yang et al., 2026) and shared-memory looped transformers (Monea et al., 2026) let later loops read the first loop's KV plus a short local window. All run the same number of loops for prefill and decode. Ouro (Zhu et al., 2025) reuses the last loop's KV for generated tokens but loses more than 10 GSM8K points when prefill does the same; our adapter is trained to read prompt KV across loops. FlashLoop (Yang and Liu, 2026) speeds up looped models without training through token-sparse loop updates, sparse attention and KV-residual quantisation.

Adaptive and phase-specific computation. Halting (Graves, 2016; Banino et al., 2021), token-level early exit (Xin et al., 2020; Elbayad et al., 2020; Schuster et al., 2022; Elhoushi et al., 2024; Corro et al., 2023) and token routing (Raposo et al., 2024) spend depth per token; CALM must also fill the skipped layers' KV for later tokens, the problem our maps solve per loop. Speculative decoding (Leviathan et al., 2023; Chen et al., 2023) pairs a cheap drafter with a deep verifier, and pause tokens add compute at prompt positions (Goyal et al., 2024). Deep-encoder/shallow-decoder models (Kasai et al., 2021; Bergner et al., 2024; S et al., 2024) spend more depth on the prompt across separate networks; SwiftKV (Qiao et al., 2025) instead skips later layers for prompt tokens. Continuous Depth Batching (Schwethelm et al., 2026) prefills at full depth, exits early during decode and studies caches shared across depths. LoopDuet (Du et al., 2026), concurrent with us, sets the two loop budgets of Huginn without training; Huginn was trained with random loop counts, whereas fixed-depth Ouro fails without training (Table 5). Position-Aware Depth Decay (Fan et al., 2025a) skips more layers for later tokens; OverFill (Kim et al., 2025) decodes with a width-pruned model; Fixed Points Part II (Huang et al., 2026) distils a light prefill network; POP (He et al., 2026), SPEED (Oh et al., 2026) and SEED (Ma et al., 2026) prefill shallower than they decode; Decode-Branch Transformers (Liu et al., 2026), PHD (Wu et al., 2025b) and State-Prediction Separation (Langford et al., 2026) add decode-side computation. Serving systems already run the two phases on separate hardware (Zhong et al., 2024; Patel et al., 2024; Qin et al., 2025) or co-schedule them (Agrawal et al., 2024) over paged KV memory (Kwon et al., 2023); untying the loops lets each phase also use its own depth. We vary prefill and decode loops independently within one loop adapter and measure what each buys.

Asymmetric KV producers and consumers. YOCO (Sun et al., 2024) has a self-decoder produce one global KV cache that a cross-decoder reads; YOCO-U (Sun et al., 2026) loops the self-decoder, keeping KV storage independent of the loop count while every token still runs the loops; KITE (Hu et al., 2026) expands a checkpoint into two towers so that bulk prefill runs only the KV-producing tower. These architectures are built for the split; we adapt an existing looped model after pretraining and choose the prefill loops m and decode loops n at inference.

Sharing and transferring state. KV is shared across heads (Shazeer, 2019; Ainslie et al., 2023) and layers (Brandon et al., 2024; Zuhri et al., 2025), and evicted or compressed per token (Xiao et al., 2024; Zhang et al., 2023; Li et al., 2024; Cai et al., 2024), which is orthogonal to and composable with our per-loop reads. MELT (Vendrell et al., 2026) post-trains Ouro to keep one gated KV cache per layer shared by all loops plus a small per-loop sliding-window cache, decoupling memory from depth; it does not separate prefill from decode loops or map which depths a task needs. Looped Latent Attention (Neill and Reid, 2026) compresses KV across loops; BeNice! (Prairie et al., 2026) and LSTM-style looped transformers (Kirsch, 2026) share it; PLatour (Shen et al., 2026) injects a larger model's prefill states into a smaller decoder; Cache-to-Cache (Fu et al., 2026), HeteroFold (Yun et al., 2026), Swarm Telepathy (Alabdulkareem et al., 2026) and CacheLink (Qiu et al., 2026) transfer KV between models, and BellowsKV (Zhang and Chen, 2026) shares one latent across layers. Like CacheLink, we train our maps through the receiver's output; within one model tokens, layers and positions align, so a RoPE-commuting scale and a value map suffice. For hybrid attention / linear-recurrence models (Yang et al., 2025; Dong et al., 2025), state-swap interventions (Afendulev et al., 2026) and LatentPort (Villani, 2026) study the two kinds of state; Appendix Q relates them to prefill loops.

G Training Details

Loop adapter. Rank-16 LoRA ($\alpha = 16$, no dropout) on the query, key, value and output projections, active only when decoding; per (decode loop, layer, KV head) a complex scale for each rotary frequency pair on keys and a full $D \times D$ value map, both initialised to the identity. Trainable parameters: 63,307,776 for Ouro-2.6B. The backbone is frozen.

Objective and data. $\mathrm{KL}(\text{teacher} \,\|\, \text{student})$ summed over answer positions and normalised by $\sum_i (|a_i| - 1)$ over the answers a_i of the global batch (one less than the supervised positions per answer, as the released checkpoints were trained); teacher = native 4/4 with the same weights; one pair (m, n) sampled per micro-batch. 48M answer tokens: OpenThoughts reasoning traces 45%, Tulu SFT responses 40%, LongAlpaca long-context answers 5% and FineWeb-Edu windows 10% (fractions of answer tokens); prompts up to 7,680 tokens, answers up to 1,024; the token budget counts $|a| - 1$ positions per answer a for the loop adapter and the distilled controls alike. Base models see a plain "User: … Assistant:" format; thinking models see the same samples in their chat template. No benchmark data was deliberately included; we did not decontaminate the training corpora. An audit afterwards (`overlap_audit.py`) finds a 13-token span shared with the training data in 187 of 542 code prompts (HumanEval+ and MBPP+), 101 of 500 MATH-500 questions, 106 of 14,042 MMLU, 75 of 6,511 BBH, 24 of 1,400 MMLU-Pro and at most 3 items of each other audited benchmark (GSM8K, ARC-Challenge, HellaSwag, Winogrande, GPQA, IFEval). Items whose extracted question text is unavailable or shorter than 13 tokens cannot be assessed (2,696 MMLU, 296 HellaSwag, 254 BBH, 247 ARC-Challenge, 194 MMLU-Pro, at most 4 elsewhere) and are kept; RULER and HELMET are synthetic or long-document and not audited. Without the overlapping items, Ours at prefill 4 / decode 1 stays significantly above Distilled 1/1 (Table 5) on MMLU ($+14.3$), MMLU-Pro ($+18.2$), BBH ($+12.4$), GSM8K ($+4.6$), HumanEval+ ($+10.0$ on the 70 remaining problems) and MBPP+ ($+5.3$), and at decode 2 above Distilled 2/2 on MMLU ($+6.3$), MMLU-Pro ($+7.7$) and BBH ($+2.6$); 4-shot MATH-500 keeps the opposite sign (Appendix H). For the per-benchmark choice of m (Table 9), the chosen m still wins significantly on 13 of 15 and 9 of 15 settings at one and two decode loops; at one decode loop MBPP+, which loses 93 of its 378 problems, becomes inconclusive ($+2.5$ $[-1.1, 10.2]$).

Optimisation. AdamW ($\beta = (0.9, 0.95)$, no weight decay), peak learning rate 10^{-4} with 2% linear warm-up and cosine decay to 10^{-5}, gradient clipping at 1.0, global batch 24 sequences on 4 B300 GPUs (6.75 hours for Ouro-2.6B), bfloat16 backbone.

Shared answer KV cache. Generated tokens keep one KV cache per layer shared by all decode loops; at inference the last decode loop commits each token, and earlier loops read that history plus the current token's KV computed live in their own

loop (*self-live*[1], a layout also used by LSTM-style looped transformers (Kirsch, 2026)). For example, when loop 1 processes token 5, it attends to tokens 1–4 through the last loop's stored KV and to token 5 through its own, freshly computed KV. Training uses the same KL objective in this layout, in two passes: a no-grad pass in which each loop reads its own causal answer KV records the last loop's answer KV; a second pass reads, at every loop, the prompt KV cache, that recorded history strictly before the current position, and the current token's KV computed live on the diagonal. Training thus approximates, but does not exactly reproduce, inference, where the history is generated recursively in the shared cache. With prompt and answer each stored once, total KV is $b(\ell + t)$, that of a one-loop model; with 1K-token prompts and 2,048 generated tokens the largest batch that fits grows from 32 (Original 4/4) to 192 and decode-4 throughput from 219 to 476 tokens/s (759 vs. 586 at n=2), at a cost of 3.0 GSM8K points at 4/4 against per-loop answer caches (Appendix O, Other design variants). On Ouro-2.6B-Thinking, a loop adapter trained the same way changes full-set GSM8K at Ours 4/2 by $+0.5$ $[-0.5,\ 1.6]$ against per-loop answer caches; at batch 1 its observed cost per generated token is similar (82.5 vs. 83.1 ms) but its answers are longer (1,166 vs. 741 tokens), so it takes $1.53\times$ $[1.27,\ 1.84]$ the time per answer. Its benefit is memory and batch size, not batch-1 latency.

SenseNova-Loop-9B. Rank-16 LoRA on the attention projections of the looped layers (full attention: q/k/v/o; Gated-DeltaNet: input and output projections), active only when decoding; no maps. The one-visit decoder reads the prefill state of the last of $m \in \{1, 2\}$ visits (sampled 50/50), so the same decoder serves prefill 1 and prefill 2. The untrained condition passes the last visit's full-attention KV and Gated-DeltaNet convolution and recurrent states. Teacher: native 2/2; 20M answer tokens of the same mixture in the SenseNova chat template; peak learning rate 2×10^{-4}.

H Evaluation Details

Protocol. Base models follow the base-model protocol of the Ouro models (Zhu et al., 2025, Appendix C.1), run through our runner so that every (m, n) pair is evaluated the same way: lm-evaluation-harness for MMLU (5-shot, log-likelihood, 14,042), MMLU-Pro (5-shot CoT, strict match; 30 per subject, 420), BBH (3-shot CoT, strict match; 60 per subtask, 1,620), ARC-Challenge (25-shot, 1,172), HellaSwag (10-shot, 10,042), Winogrande (5-shot, 1,267) and GSM8K (3-shot CoT, strict match, 1,319); MATH-500 with the 4-shot Minerva prompt, scored on the final answer (the last `\boxed{}`, else the Minerva closing sentence) with math-verify (the Ouro models' own 5-shot prompt and scorer are not released); HumanEval+ and MBPP+ with evalplus as it runs for a tokenizer with a chat template (the problem in evalplus's chat prompt, greedy, 768 new tokens, sanitized code, plus tests). With this protocol native Ouro-2.6B 4/4 scores within one point of the published numbers on MMLU, MMLU-Pro, BBH, ARC-Challenge and HellaSwag, 1.9 above on Winogrande and within 1.2 to 3.1 points on HumanEval(+) and MBPP(+) (Table 7). Two scores differ for a documented reason: GSM8K strict matching requires the exemplars' "The answer is" sentence and gives 69.4, while flexible extraction of the last number gives 86.7, bracketing the published 81.6; and MATH-500 with the public 4-shot Minerva prompt gives 53.6 against 90.9 with Ouro's unreleased 5-shot prompt and scorer. Every configuration in this paper is scored the same way, but a protocol can change the size of a difference, not only the level. Rescoring the same GSM8K answers with flexible extraction, the aligned adapter at prefill 4 / decode 1 gains more over LoRA-distilled 1/1 ($+6.7$ $[+4.0, +9.3]$ instead of $+4.5$), while at decode 2 its gain over native 2/2 shrinks from $+3.4$ $[+1.2, +5.6]$ to $+1.2$ $[-0.8, +3.3]$.

GPQA-Diamond (0-shot), IFEval (prompt-level strict), RULER (13 tasks $\times$ 40 rows per length) and HELMET (RAG, synthetic recall, re-ranking and ICL; 1,108 rows) are prepared and scored by our own scripts. Thinking models answer GSM8K and MATH-500 zero-shot in their chat template, answers in `\boxed{}` scored with math-verify, with budgets of 4,096 and 8,192 tokens, greedily rather than with the sampling and model-judged scoring of their original evaluation; their MMLU-Pro and BBH are as for base models and their code as above. SenseNova-Loop-9B uses MMLU-Pro with 70 questions per subject (980) and the full BBH (6,511) through our own scripts. On raw HELMET prompts thinking checkpoints

[1] Self-live was developed by the author in separate work on shared KV caches for looped models; here it is applied to the loop adapter's answer tokens.

benchmark	ours	reported	items
MMLU	74.6	74.6	14,042
MMLU-Pro	56.1	55.7	1,400
BBH	80.7	80.5	6,511
ARC-C	66.5	66.4	1,172
HellaSwag	79.5	79.7	10,042
Winogrande	77.7	75.9	1,267
GSM8K	69.4	81.6	1,319
GSM8K (flexible extraction)	86.7	–	1,319
MATH-500	53.6	90.9	500
HumanEval	75.6	78.7	164
HumanEval+	69.5	70.7	164
MBPP	77.5	80.4	378
MBPP+	64.6	66.6	378

Table 7: **Reproducing the native model.** Ouro-2.6B 4/4 under our evaluation and as reported by Zhu et al. (2025, Table 8); MMLU-Pro and BBH here on the full runs of the native model.

can emit a newline and stop; for thinking models we therefore mask the stop tokens until an answer has a non-whitespace token, for every configuration. Intervals: paired bootstrap over items, 5,000 resamples, recomputed from released per-item scores by one script (Appendix S).

Scoring and prompts. lm-evaluation-harness's strict Minerva match for MATH-500 credits an answer only if it ends with the exemplars' sentence "Final Answer: The final answer is X. I hope it is correct." Native Ouro-1.4B 4/4 writes boxed answers that are mostly correct but closes them in other words, so it scores 4.4 under strict matching and 70.6 on the final answer; on Ouro-2.6B the decoder with one prefill loop and four decode loops writes longer solutions than the exemplars, ends with that sentence on 183 of 500 problems (429 with four prefill loops), and scores 18.6 under strict matching but 68.2 on the final answer (51.0 with four prefill loops). We therefore score the final answer; the prefill-1 advantage on 4-shot MATH-500 is in the answers, not in their format. Code is different: under an earlier lm-evaluation-harness prompt, prefilling base Ouro-2.6B with two or more loops dropped Python indentation in 35–63% of MBPP+ answers (under 3% with one), and that task scored only the first base assertion; with evalplus this effect is gone. Answer format also changes one comparison of Appendix L: for Nanbeige4.2-3B-Base, Ours 2/1 minus Ours 1/1 on GSM8K is -8.4 with strict matching but positive with flexible extraction. Native throughput baselines: static per-loop caches with the same two-call flash attention (fastest), the standard dynamic cache (500 / 245 / 105 tokens/s at $n = 1/2/4$), and a preallocated contiguous cache with one attention call (267 / 130 / 57); all produce identical greedy tokens.

I Two-Seed Checks on Selected Comparisons

	task	seed 1	seed 2
prefill 4 − prefill 1 (decode 1)	GSM8K	$+8.5$	$+8.9$
	MMLU	$+15.2$	$+15.2$
	RULER-4K	-3.8	-3.5
	RULER-8K	$+2.5$	$+2.0$
	RULER-16K	$+8.6$	$+8.6$
prefill 4 − 2 (decode 2)	GSM8K	-0.1	-1.1
	RULER-4K	-7.2	-6.8

Table 8: Ouro-2.6B loop adapter trained with two seeds (initialisation and (m, n) sampling); paired differences within each seed, full sets.

J Choosing the Prefill Loops per Benchmark

Table 9 chooses the prefill count per benchmark on held-out halves of its items and compares the choice with native n/n; Table 12 compares the same choice with native n/n distilled on the same data.

benchmark	decode 1	decode 2	decode 3	decode 4
MMLU	$+15.1^{*}$ [14.2, 16.0] (3,4)	$+5.6^{*}$ [4.8, 6.3] (4)	$+0.6$ [−0.2, 1.1] (4)	-0.5^{*} [−1.1, −0.2] (4)
GSM8K	$+13.2^{*}$ [9.2, 15.7] (2)	$+5.7^{*}$ [1.9, 7.7] (3)	$+0.5$ [−1.9, 2.3] (3)	$+1.0$ [−1.7, 2.7] (3)
MATH-500 4-shot	$+5.6^{*}$ [1.8, 9.2] (1)	$+14.4^{*}$ [10.2, 18.4] (1)	$+14.2^{*}$ [10.0, 18.4] (1)	$+14.6^{*}$ [10.4, 18.8] (1)
BBH 3-shot	$+20.9^{*}$ [17.1, 23.6] (4)	$+6.5^{*}$ [3.0, 8.8] (3)	-2.3 [−3.9, 0.4] (4,2)	-1.7^{*} [−4.3, −0.2] (3)
ARC-C	$+14.8^{*}$ [10.6, 17.2] (4)	$+2.7^{*}$ [0.5, 4.4] (4,3)	$+0.9$ [−1.0, 1.9] (3)	-0.2 [−2.5, 0.7] (3)
MMLU-Pro	$+19.0^{*}$ [13.6, 25.2] (4,3)	$+5.7$ [−0.5, 10.5] (4,3)	$+0.7$ [−5.2, 5.0] (4)	-2.1 [−8.3, 2.6] (4,3)
HellaSwag	$+3.9^{*}$ [3.1, 4.6] (3)	$+0.5$ [−0.2, 0.9] (3)	$+0.0$ [−0.4, 0.4] (3,4)	$+0.0$ [−0.5, 0.4] (3,4)
Winogrande	$+6.2^{*}$ [2.2, 9.2] (3)	$+2.2^{*}$ [0.1, 5.1] (3,4)	$+0.1$ [−2.7, 1.7] (2)	-0.9 [−3.2, 0.6] (2)
HumanEval+	$+21.3^{*}$ [11.6, 28.7] (3)	$+7.9$ [0.0, 12.8] (3)	$+1.2$ [−6.7, 5.5] (3)	$+3.7$ [−3.0, 7.9] (3)
MBPP+	$+4.2^{*}$ [0.8, 9.8] (4,2)	$+2.4$ [−1.9, 5.0] (2)	-0.5 [−3.7, 2.1] (2,3)	-0.5 [−6.1, 2.4] (2)
GPQA-Diamond	$+12.1$ [−2.5, 18.7] (4)	$+3.5$ [−7.1, 11.6] (3,4)	-1.0 [−9.1, 7.6] (4,2)	-3.5 [−10.6, 4.5] (3,4)
IFEval	$+9.4^{*}$ [4.1, 12.8] (4)	-0.9 [−5.5, 3.5] (4,3)	$+1.8$ [−2.8, 4.8] (2)	$+0.4$ [−4.3, 4.3] (4,2)
RULER-4K	$+12.3^{*}$ [6.2, 15.9] (2)	$+5.7^{*}$ [0.3, 7.9] (2)	-1.6^{*} [−6.1, −0.01] (2)	-2.3^{*} [−6.3, −0.6] (2)
RULER-8K	$+17.6^{*}$ [10.4, 21.6] (3)	$+5.6^{*}$ [2.4, 10.6] (2,3)	-9.0^{*} [−11.8, −5.5] (2,3)	-10.3^{*} [−13.2, −6.0] (2,3)
RULER-16K	$+19.6^{*}$ [15.9, 25.2] (4,3)	$+10.4^{*}$ [5.8, 15.4] (4,3)	-5.6^{*} [−9.8, −2.0] (3,4)	-5.2^{*} [−9.8, −2.6] (3)

Table 9: **Choosing the prefill loops per benchmark, against the native model with the same decode loops.** Ouro-2.6B, one loop adapter. For each benchmark and number of decode loops n we choose the prefill count m on one half of the items, score it on the other half, and swap, so that no item is scored by a choice made on it; the table gives the resulting accuracy minus native n/n on the same items, the m chosen on each half (in parentheses), and paired 95% bootstrap intervals that repeat the choice of m in every resample (*: interval excludes zero; nominal, per benchmark). Score points on a 0–100 scale.

K Full Task Map

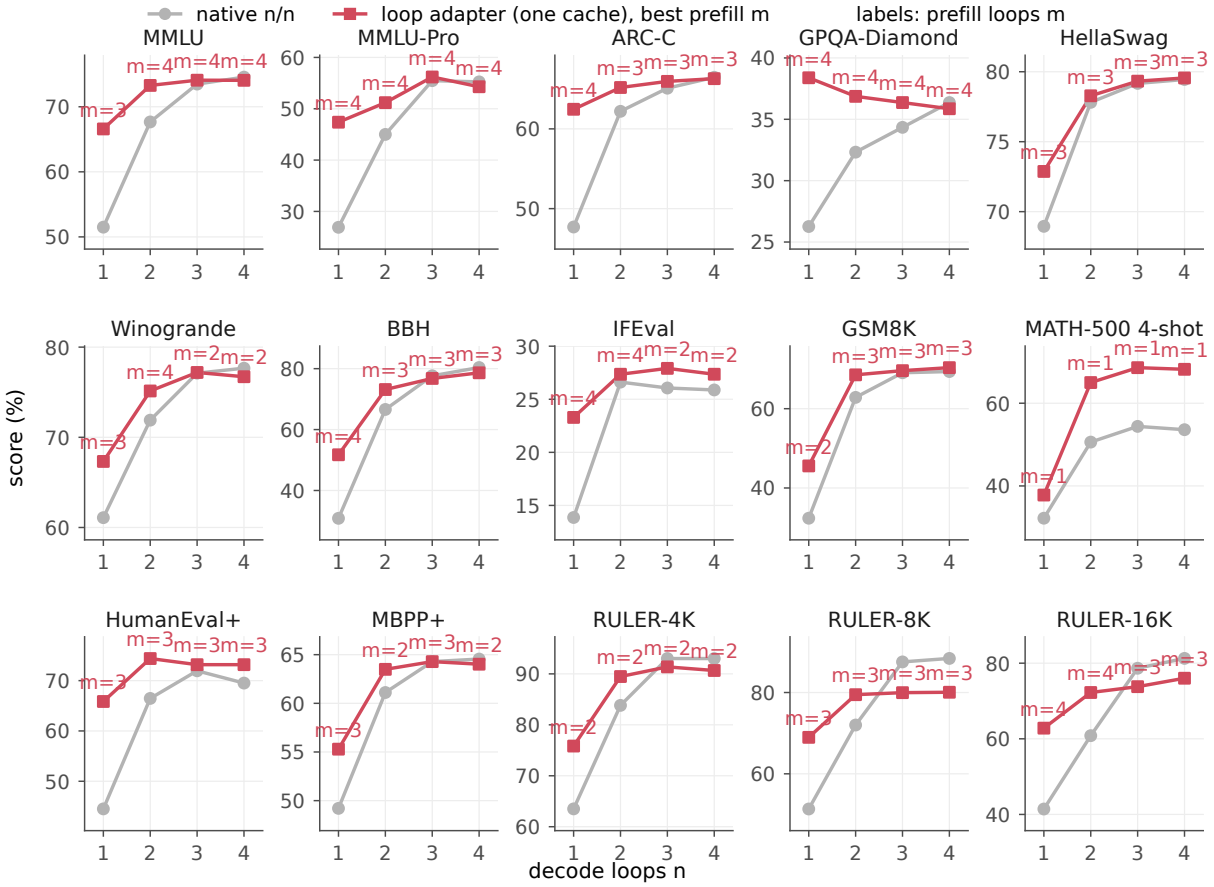


Figure 12: **All 15 task-map settings.** Ouro-2.6B, one loop adapter with one cache; grey: native n/n; red: the best prefill count m at each n (labels; the smallest maximizing m), chosen on these items; the held-out choice is Table 9. Every cell: Table 10.

	MMLU	GSM8K	MATH	BBH	ARC-C	MMLU-P	HSwag	Wino	HE+	MBPP+	GPQA	IFEv	R-4K	R-8K	R-16K
native 1/1	51.5	32.4	32.2	30.9	47.7	26.9	69.0	61.1	44.5	49.2	26.3	13.9	63.5	51.3	41.4
1/1	51.4	36.2	**37.8**	37.9	52.6	27.9	69.2	65.8	51.2	48.4	26.3	21.6	74.1	64.3	54.2
2/1	63.5	**45.6**	25.6	50.3	59.7	43.1	72.5	66.3	59.1	54.2	30.3	22.0	**75.8**	66.9	61.6
3/1	**66.6**	43.3	25.4	51.4	61.5	46.2	**72.9**	**67.3**	**65.9**	**55.3**	32.8	21.4	74.0	**69.0**	61.3
4/1	66.6	44.7	25.8	**51.7**	**62.5**	**47.4**	72.4	67.1	64.6	55.3	**38.4**	**23.3**	70.2	66.8	**62.8**
native 2/2	67.7	62.9	50.6	66.6	62.2	45.0	77.8	71.9	66.5	61.1	32.3	26.6	83.8	72.0	60.8
1/2	53.3	56.6	**65.0**	56.5	58.2	33.1	74.7	68.3	64.6	56.9	26.8	22.7	77.9	69.7	57.6
2/2	67.3	66.8	49.6	71.4	64.1	47.4	77.8	74.2	64.6	**63.5**	34.3	25.7	**89.5**	78.3	66.9
3/2	72.9	**68.5**	48.4	**73.1**	**65.2**	51.0	**78.3**	74.4	**74.4**	63.0	36.4	26.6	85.5	**79.5**	71.6
4/2	**73.3**	66.7	47.0	71.2	65.1	**51.2**	78.0	**75.1**	68.3	62.7	**36.9**	**27.4**	82.3	77.9	**72.2**
native 3/3	73.5	69.1	54.4	77.7	65.1	55.5	79.2	77.1	72.0	64.3	34.3	26.1	93.0	87.5	78.7
1/3	53.0	60.5	**68.6**	64.0	58.5	32.6	76.0	68.5	66.5	59.3	27.3	22.4	80.3	71.5	58.1
2/3	67.4	68.8	53.4	76.3	64.8	49.3	79.1	**77.2**	70.1	63.8	34.3	**27.9**	**91.4**	79.2	69.3
3/3	73.5	**69.6**	53.2	**76.8**	**66.0**	53.3	**79.3**	75.5	**73.2**	**64.3**	34.3	26.8	87.2	**80.0**	**73.8**
4/3	**74.1**	67.6	52.2	76.5	65.4	**56.2**	79.2	76.1	68.3	62.2	**36.4**	25.7	82.8	77.9	73.4
native 4/4	74.6	69.4	53.6	80.4	66.5	55.2	79.4	77.7	69.5	64.5	36.4	25.9	93.0	88.4	81.3
1/4	52.9	60.7	**68.2**	64.5	57.9	33.1	76.2	69.1	69.5	59.0	29.8	24.6	80.4	70.1	57.0
2/4	67.5	69.0	54.8	77.0	65.2	47.9	79.1	**76.7**	67.7	**64.0**	34.3	**27.4**	**90.7**	78.2	68.3
3/4	73.5	**70.4**	54.8	**78.6**	**66.3**	53.6	**79.6**	75.5	**73.2**	63.8	32.8	26.8	87.6	**80.1**	**76.0**
4/4	**74.1**	68.8	51.0	78.0	65.3	**54.3**	79.5	75.7	66.5	62.7	**35.9**	27.2	82.3	76.6	73.4

Table 10: **Every cell of the task map, one cache.** Ouro-2.6B, accuracy (%) of native n/n and of one loop adapter with one cache at prefill m / decode n (m / n), each column on the items that all 20 configurations share; bold: the best m at each n, chosen on these items (Figure 12; the held-out choice is Table 9). MMLU and GSM8K are the full sets (14,042 and 1,319 items); RULER uses 520 items per length. MMLU-P: MMLU-Pro, HEval: HumanEval, IFEv: IFEval, R-kK: RULER at kK tokens.

	MMLU	MMLU-P	BBH	ARC-C	HSwag	Wino	GSM8K	MATH	HE+	MBPP+	R-16K
native 1/1	51.5	26.9	30.9	47.7	69.0	61.1	32.4	32.2	44.5	49.2	41.4
1 / 1	51.2	26.2	36.5	52.5	69.0	65.3	35.1	**34.8**	46.3	49.2	52.7
2 / 1	63.2	41.7	50.0	60.0	72.3	67.1	44.9	26.4	59.1	54.8	60.2
3 / 1	66.4	**46.2**	52.0	61.8	**73.1**	**68.2**	44.7	25.2	**65.9**	**56.3**	62.4
4 / 1	**67.1**	45.0	**52.7**	**62.8**	72.7	68.0	**45.0**	25.4	65.2	**56.3**	**64.7**
native 2/2	67.7	45.0	66.6	62.2	77.8	71.9	62.9	50.6	66.5	61.1	60.8
1 / 2	53.4	31.9	55.7	58.5	74.6	68.0	55.9	**63.8**	61.6	56.3	57.1
2 / 2	67.6	47.4	69.1	62.4	77.5	73.1	64.7	51.8	66.5	61.6	68.6
3 / 2	72.9	53.8	**73.7**	64.3	78.1	**75.8**	**68.7**	48.0	**73.2**	63.2	**76.1**
4 / 2	**73.5**	**54.0**	73.5	**65.0**	**78.3**	75.1	66.3	45.8	70.1	**64.3**	73.5
native 3/3	73.5	55.5	77.7	65.1	79.2	77.1	69.1	54.4	72.0	64.3	78.7
1 / 3	53.0	32.6	63.1	58.3	76.0	68.4	60.0	**68.8**	67.1	58.2	58.8
2 / 3	67.4	46.2	74.0	63.5	78.4	74.3	66.2	62.4	68.3	60.6	68.6
3 / 3	73.6	53.8	**77.9**	65.0	79.1	**76.7**	**69.3**	55.8	**72.6**	63.5	79.2
4 / 3	**74.3**	**56.7**	77.7	**66.2**	**79.2**	76.3	68.4	53.0	69.5	**63.8**	**79.8**
native 4/4	74.6	55.2	80.4	66.5	79.4	77.7	69.4	53.6	69.5	64.5	81.3
1 / 4	53.0	32.9	64.8	58.0	76.4	69.1	61.2	**69.6**	67.7	59.8	57.8
2 / 4	67.7	45.5	74.3	62.6	78.4	74.2	67.0	64.8	67.7	62.2	66.7
3 / 4	73.6	**53.8**	77.8	65.4	79.1	76.5	**68.8**	59.6	**71.3**	**63.8**	79.1
4 / 4	**74.5**	**53.8**	**79.1**	**66.1**	**79.4**	**77.0**	**68.8**	54.6	68.9	63.2	**81.0**

Table 11: **Every cell of the task map, aligned caches.** Ouro-2.6B, accuracy (%) of native n/n and of the aligned loop adapter at prefill m / decode n on the benchmarks of Table 5, each column on the items all available configurations share (Figure 5); bold: the best m at each n.

benchmark	decode 1	decode 2	decode 3
MMLU	$+13.8^{*}$ [12.9, 14.8]	$+6.1^{*}$ [5.3, 6.8]	$+1.0^{*}$ [0.3, 1.5]
GSM8K	$+5.2^{*}$ [1.3, 7.6]	-1.1 [-4.8, 0.9]	-1.7 [-4.2, 0.0]
MATH-500 4-shot	$+0.8$ [-3.0, 4.4]	$+18.2^{*}$ [14.0, 22.4]	$+14.8^{*}$ [10.6, 19.0]
BBH 3-shot	$+11.3^{*}$ [7.4, 14.2]	$+2.7$ [-0.7, 4.6]	-2.3 [-3.8, 0.4]
ARC-C	$+7.5^{*}$ [3.7, 9.6]	$+2.0$ [-0.1, 3.7]	$+0.5$ [-1.3, 1.5]
MMLU-Pro	$+19.3^{*}$ [14.0, 25.5]	$+4.5$ [-1.9, 9.3]	$+2.9$ [-2.6, 6.9]
HellaSwag	$+2.7^{*}$ [2.1, 3.2]	$+0.7^{*}$ [0.1, 1.1]	-0.0 [-0.4, 0.4]
Winogrande	$+0.9$ [-2.8, 3.4]	-0.5 [-2.4, 2.2]	$+0.1$ [-2.6, 1.7]
HumanEval+	$+11.6^{*}$ [2.4, 18.3]	$+4.9$ [-2.4, 9.8]	$+4.3$ [-3.7, 9.1]
MBPP+	$+3.2$ [-0.8, 9.0]	$+0.5$ [-3.7, 3.2]	-0.3 [-3.7, 2.7]
GPQA-Diamond	$+10.6$ [-3.5, 17.2]	$+4.5$ [-5.6, 11.1]	$+1.0$ [-5.6, 9.1]
IFEval	$+2.2$ [-3.0, 5.2]	$+0.7$ [-4.1, 5.0]	$+1.5$ [-3.1, 4.4]
RULER-4K	-3.5^{*} [-9.4, -0.4]	-1.4^{*} [-6.5, -0.1]	-1.8^{*} [-6.3, -0.2]
RULER-8K	$+0.8$ [-6.2, 4.1]	-5.4^{*} [-8.2, -0.7]	-8.7^{*} [-11.5, -5.3]
RULER-16K	$+6.3^{*}$ [3.1, 11.6]	$+0.9$ [-3.3, 5.5]	-8.0^{*} [-12.1, -4.5]

Table 12: **At one decode loop, choosing the prefill loops beats distilled native models on knowledge, code and few-shot math.** Ouro-2.6B: the m chosen as in Table 9, minus native n/n distilled with LoRA (same data, teacher and 48M tokens as the loop adapter, supervising every answer token as the adapter does), on the same items; selection-aware paired 95% intervals (*: excludes zero). At two decode loops the counts are mixed (Section D.2); at four decode loops the distilled model is the teacher itself.

L Other Checkpoints

Table 15 compares prefill counts at one decode loop within the same loop adapter on five checkpoints from two model families.

Table 13 repeats the per-benchmark choice on Ouro-1.4B. As on Ouro-2.6B, the gain is largest with one decode loop (significant on 12 of 15 settings) and shrinks with more (2 of 15 at two decode loops, where 3-shot BBH is significantly worse than Original 2/2); within the adapter the chosen m beats the adapter's own n/n significantly on 6 and 1 of 15. Against native 1/1 and 2/2 distilled with LoRA on the same data, teacher and tokens, it is significantly above on 6 and below on 3 of 15 at one decode loop (below on all three RULER lengths, where distillation alone helps more), and above on 4 and below on 4 at two. With fixed configurations (Table 14), the aligned adapter at 4/1 averages 43.9 against 34.5 for native 1/1 and 40.0 for native 1/1 distilled on the same data; at two decode loops the distilled 2/2 is stronger on BBH, MATH-500 and RULER-16K, so the smaller model gains less than Ouro-2.6B. Nanbeige4.2-3B-Base, a non-looped model run with its blocks repeated and one loop adapter trained per (m, n) pair, shows the same direction at one decode loop: Ours 2/1 minus Ours 1/1 is $+11.8$, $+11.7$ and $+14.2$ on RULER 4K/8K/16K and $+12.2$ on MMLU (GSM8K: Appendix H).

benchmark	decode 1	decode 2	decode 3	decode 4
MMLU	$+23.4^{*}$ [19.2, 26.7] (3)	$+5.0^{*}$ [2.5, 7.8] (4,3)	$+0.3$ [−1.4, 2.6] (3,4)	-0.2 [−1.5, 1.5] (3,4)
GSM8K	$+9.0^{*}$ [4.8, 14.8] (2,4)	$+6.2$ [0.0, 10.8] (3)	$+0.2$ [−6.2, 4.2] (3)	$+2.8$ [−5.0, 6.5] (3)
MATH-500 4-shot	$+2.6$ [−2.2, 7.0] (3,1)	$+0.2$ [−7.2, 3.8] (3)	-6.4^{*} [−9.8, −1.4] (3,2)	-2.0 [−8.4, 1.2] (1)
BBH 3-shot	$+12.8^{*}$ [10.6, 15.1] (1)	-5.4^{*} [−10.7, −2.9] (1)	-4.0^{*} [−7.8, −1.9] (3)	-2.9^{*} [−6.5, −1.0] (3)
ARC-C	$+17.2^{*}$ [13.8, 20.1] (4)	$+4.2^{*}$ [2.0, 6.5] (3,4)	$+1.1$ [−0.9, 2.6] (4)	-0.1 [−2.0, 1.1] (4)
MMLU-Pro	$+17.9^{*}$ [10.7, 23.1] (3)	$+6.2$ [−1.2, 10.5] (3)	$+2.9$ [−2.6, 6.7] (3)	$+0.5$ [−3.8, 4.8] (4,3)
HellaSwag	$+8.0^{*}$ [7.0, 8.9] (3)	$+0.3$ [−0.4, 0.9] (3)	-0.2 [−0.7, 0.2] (3)	-0.6^{*} [−1.1, −0.2] (3)
Winogrande	$+6.8^{*}$ [2.5, 10.3] (3,2)	$+2.3$ [−1.0, 4.7] (2)	-2.8^{*} [−5.1, −0.4] (3,2)	-1.1 [−4.1, 0.5] (3)
HumanEval+	$+5.5$ [0.0, 14.0] (2,4)	$+5.5$ [−2.4, 13.4] (4,3)	$+0.6$ [−4.9, 9.1] (4,1)	-3.7 [−7.9, 4.3] (4,1)
MBPP+	$+9.0^{*}$ [1.9, 13.2] (3)	$+0.5$ [−5.3, 4.2] (3)	$+2.4$ [−3.4, 5.0] (3)	$+2.1$ [−2.1, 4.5] (3)
GPQA-Diamond	-3.5 [−11.6, 11.1] (3,1)	$+6.1$ [−2.5, 15.2] (3,4)	-7.1 [−14.6, 0.5] (3,4)	-3.5 [−13.1, 0.5] (4)
IFEval	$+7.2^{*}$ [3.0, 12.4] (2,4)	-1.1 [−5.0, 3.1] (2,3)	$+0.4$ [−6.5, 3.7] (2)	-3.0 [−6.3, 1.3] (2,3)
RULER-4K	$+14.5^{*}$ [11.0, 19.3] (1,3)	$+4.1$ [−2.7, 7.7] (1)	-11.7^{*} [−20.0, −8.4] (1)	-13.2^{*} [−22.2, −9.8] (1)
RULER-8K	$+14.5^{*}$ [9.2, 18.2] (2)	$+0.9$ [−2.8, 5.5] (2,1)	-12.0^{*} [−17.7, −8.7] (1)	-17.8^{*} [−21.4, −11.6] (3,1)
RULER-16K	$+13.4^{*}$ [9.9, 17.6] (2,4)	$+1.5$ [−3.3, 5.6] (4)	-6.2^{*} [−10.4, −1.4] (1,4)	-18.4^{*} [−22.1, −13.0] (1,3)

Table 13: **Table 9 on Ouro-1.4B** (its own loop adapter; the 15 settings of Table 9; MMLU and GSM8K subsets of 20 questions per subject and 400 problems): prefill loops chosen on one half of the items, scored on the other, minus native n/n on the same items, in points; the m chosen on each half in parentheses; paired 95% bootstrap intervals that repeat the choice of m in every resample (*: excludes zero).

n	method	MMLU	MMLU-Pro	BBH	ARC-C	HellaSw.	Winogr.	GSM8K	MATH	HE+	MBPP+	RULER-16K	Avg.
1	Original 1/1	42.0	19.1	19.9	38.6	55.3	56.0	17.2	**24.2**	34.1	41.3	31.6	34.5
1	Training-free 4/1	32.8	0.0	0.0	31.7	27.8	51.1	0.0	0.0	0.0	0.0	0.0	13.0
1	Distilled 1/1	45.0	21.2	**32.8**	46.4	58.8	61.2	21.0	23.0	37.8	42.9	**49.8**	40.0
1	Ours, one cache 4/1	64.0	33.3	8.2	55.8	62.9	62.6	**28.2**	22.4	**43.3**	**48.4**	45.9	43.2
1	**Ours, aligned 4/1**	**64.0**	**36.7**	13.4	**56.3**	**63.3**	**63.1**	27.0	**24.2**	39.6	47.6	47.2	**43.9**
2	Original 2/2	62.2	39.5	50.8	54.6	**71.5**	66.5	49.8	55.8	54.9	**57.7**	50.6	55.8
2	Training-free 4/2	37.5	1.2	0.0	35.1	30.9	57.3	0.0	0.0	0.0	0.0	8.5	15.5
2	Distilled 2/2	62.5	36.0	**55.7**	55.0	70.5	67.9	50.2	**58.0**	56.7	55.8	**63.1**	57.4
2	Ours, one cache 4/2	67.7	42.9	42.8	**58.9**	71.3	**68.3**	52.2	49.8	**62.8**	54.8	52.0	56.7
2	**Ours, aligned 4/2**	**67.9**	**43.8**	49.4	58.7	71.4	67.6	**55.0**	53.2	**62.8**	57.1	53.5	**58.2**
4	Original 4/4	67.6	45.7	69.5	60.1	74.4	70.6	62.2	70.6	65.8	60.9	73.9	65.6

Table 14: **Table 5 on Ouro-1.4B**, with its own adapters and controls (MMLU 20 questions per subject and 400 GSM8K problems, as in its task map); each column on the items all rows share.

model (same decoder, decode 1)	knowledge	long context	multi-step
Ouro-2.6B, prefill 4 vs. 1	MMLU $+15.2$ [14.3, 16.1]	RULER-16K $+8.6$ [4.3, 12.8]	GSM8K $+8.5$ [6.1, 10.9]
Ouro-1.4B, prefill 4 vs. 1	ARC-C $+10.8$ [8.4, 13.1]	RULER-16K $+1.4$ [−2.6, 5.4]	GSM8K $+5.0$ [2.7, 7.2]
Ouro-2.6B-Thinking, prefill 4 vs. 1	MMLU-Pro $+18.6$ [13.6, 23.8]	HELMET $+7.7$ [2.1, 13.6]	GSM8K $+6.6$ [4.2, 9.2]
Ouro-1.4B-Thinking, prefill 4 vs. 1	MMLU-Pro $+17.1$ [12.4, 22.4]	HELMET $+4.6$ [0.1, 8.6]	GSM8K $+13.6$ [10.5, 16.6]
SenseNova-Loop-9B, prefill 2 vs. 1	MMLU-Pro $+5.5$ [2.9, 8.1]	HELMET $+3.5$ [0.8, 6.1]	BBH $+7.0$ [5.9, 8.0]

Table 15: **Prefill loops with the decoder held fixed** across five checkpoints (paired, within one loop adapter). Set sizes: MMLU 14,042, ARC-C 1,172, MMLU-Pro 420 (Ouro) and 980 (SenseNova), RULER 520, HELMET 1,108 (Ouro) and 6,646 (SenseNova), GSM8K 1,319, BBH 6,511. HELMET is averaged as its authors do (mean per dataset, then per category, then over its four categories). Each column shows one benchmark of that kind per checkpoint; counterexamples are in Appendix R. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

M Prefill 4 / Decode 2 in Numbers, and on Ouro-1.4B-Thinking

Table 16 lists the scores and paired intervals behind Figure 6 and repeats the comparison on the smaller Thinking checkpoint. The effect of prefill loops replicates: with the same loop adapter, prefilling with four loops instead of two raises MMLU-Pro ($+8.6$), HELMET ($+7.3$), ARC-Challenge ($+2.7$), BBH ($+8.8$), HumanEval+ ($+7.9$) and MBPP+ ($+6.3$), with no detectable difference on GSM8K and MATH-500. Against native 2/2 the result is weaker than on Ouro-2.6B-Thinking: prefill 4 / decode 2 is significantly better on GSM8K ($+3.0$), ARC-Challenge ($+4.3$) and HumanEval+ ($+10.4$), within noise on four bench-

	score				adapter 4/2 minus		
benchmark	native 2/2	adapter 2/2	adapter 4/2	native 4/4	native 2/2	adapter 2/2	native 4/4
Ouro-2.6B-Thinking (Figure 6)							
GSM8K	92.2	93.0	93.3	94.2	+1.1 [−0.2, 2.4]	+0.2 [−1.0, 1.4]	−1.0 [−2.0, 0.1]
MATH-500	84.8	84.8	85.0	89.8	+0.2 [−2.6, 3.0]	+0.2 [−2.8, 3.2]	−4.8 [−7.8, −2.0]
MMLU-Pro	43.1	41.9	50.0	49.0	+6.9 [3.1, 11.0]	+8.1 [4.0, 12.1]	+1.0 [−2.6, 4.5]
ARC-C	62.5	63.8	66.1	67.5	+3.6 [2.0, 5.3]	+2.3 [0.9, 3.7]	−1.4 [−2.6, −0.1]
BBH	66.6	71.3	73.5	71.7	+6.9 [4.6, 9.2]	+2.2 [0.1, 4.3]	+1.9 [−0.4, 4.1]
HumanEval+	68.9	70.7	74.4	73.8	+5.5 [−0.6, 11.6]	+3.7 [−2.4, 9.8]	+0.6 [−4.9, 6.1]
MBPP+	61.9	61.6	62.4	60.6	+0.5 [−2.9, 4.0]	+0.8 [−2.4, 4.0]	+1.9 [−0.5, 4.2]
HELMET	44.0	49.5	57.0	65.7	+13.0 [6.7, 19.5]	+7.5 [1.4, 13.8]	−8.7 [−13.3, −4.1]
Ouro-1.4B-Thinking							
GSM8K	86.4	89.5	89.5	92.3	+3.0 [1.3, 4.8]	+0.0 [−1.5, 1.4]	−2.9 [−4.3, −1.5]
MATH-500	76.4	77.6	75.6	85.2	−0.8 [−4.6, 3.0]	−2.0 [−5.8, 1.8]	−9.6 [−12.8, −6.2]
MMLU-Pro	39.8	34.8	43.3	46.2	+3.6 [−1.0, 8.1]	+8.6 [4.3, 12.9]	−2.9 [−6.7, 1.2]
ARC-C	56.1	57.7	60.4	62.5	+4.3 [2.1, 6.4]	+2.7 [1.0, 4.5]	−2.0 [−3.7, −0.4]
BBH	48.7	24.0	32.8	70.0	−15.9 [−18.9, −13.0]	+8.8 [6.2, 11.5]	−37.2 [−39.9, −34.5]
HumanEval+	59.1	61.6	69.5	73.2	+10.4 [4.3, 17.1]	+7.9 [1.8, 14.0]	−3.7 [−8.5, 1.2]
MBPP+	55.3	52.9	59.3	62.4	+4.0 [−0.3, 8.2]	+6.3 [2.4, 10.3]	−3.2 [−6.1, −0.3]
HELMET	45.0	39.2	46.5	64.9	+1.5 [−4.8, 7.8]	+7.3 [2.2, 12.7]	−18.3 [−22.8, −13.6]

Table 16: **Figure 6 in numbers, and on Ouro-1.4B-Thinking.** Per checkpoint, one loop adapter serves prefill 2 / decode 2 and prefill 4 / decode 2; same benchmarks, protocols and items for both checkpoints. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

	score			distilled 2/2	prefill 4 / decode 2
benchmark	native 2/2	distilled 2/2	prefill 4 / decode 2	minus native 2/2	minus distilled 2/2
GSM8K	92.2	93.0	93.3	+0.8 [−0.4, 2.0]	+0.2 [−1.0, 1.4]
MATH-500	84.2	83.2	83.8	−1.0 [−4.5, 2.5]	+0.5 [−2.5, 3.2]
MMLU-Pro	43.1	44.0	50.0	+1.0 [−2.9, 4.5]	+6.0 [1.7, 10.2]
ARC-C	62.5	63.0	66.1	+0.4 [−0.7, 1.5]	+3.2 [1.5, 4.8]
BBH	66.6	71.4	73.5	+4.8 [2.8, 6.9]	+2.1 [−0.1, 4.3]
HumanEval+	68.9	69.5	74.4	+0.6 [−4.3, 5.5]	+4.9 [−0.6, 11.0]
MBPP+	61.9	61.6	62.4	−0.3 [−2.6, 2.1]	+0.8 [−2.6, 4.5]
HELMET	44.0	53.8	57.0	+9.8 [6.9, 12.9]	+3.2 [−2.7, 9.3]

Table 17: **Untying versus distillation** (Ouro-2.6B-Thinking). "Distilled 2/2": rank-16 LoRA on the native two-loop model, trained with the same data, teacher (native 4/4) and number of tokens as the loop adapter. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

marks, and much worse on 3-shot BBH (−15.9): on this raw few-shot prompt the adapter often ends its answer right after the exemplars' opening "Let's think step by step." (734 of 1,620 answers at Ours 4/2, 28 for Original 2/2), an early stop that its chat-format training data never show; Ouro-2.6B-Thinking does not do this. Against native 4/4 the gap is larger than on Ouro-2.6B-Thinking, most of all on BBH (−37.2) and HELMET (−18.3).

N End-to-End Time and Decode Throughput

Serving. Storing the prompt once also changes throughput (Figure 7; Table 19 in Appendix N). In a controlled decode benchmark (8K context, 32 repeated decode steps, largest batch that fits on one B300), one prompt KV cache decodes 0.97× as fast as the fastest native implementation we measured at one decode loop and 1.19–1.62× at two to four, because it admits larger batches as n grows; this native baseline uses the same flash attention and is faster than the standard dynamic cache (Appendix H). With long answers (1K-token prompts, 2,048 generated tokens) one prompt KV cache decodes 0.97× native at n=2 and 1.16× at n=4.

task / model	config	acc. (%)	s / answer	tokens	N
GSM8K Ouro-2.6B-T	Original 2/2	92.7	63.3	991	192
	Original 4/4	92.7	118	878	192
	distilled 2/2	94.3	57.7	837	192
	Ours 2/2	95.8	68.3	788	192
	Ours 4/2	94.3	69.6	788	192
MATH-500 Ouro-2.6B-T	Original 2/2	85.0	243	3191	40
	Original 4/4	82.5	469	3069	40
	Ours 4/2	85.0	298	3519	40
GSM8K SenseNova	Original 2/2	98.4	48.0	365	64
	Ours 1/1	96.9	28.4	380	64
	Ours 2/1	95.3	30.6	380	64

Table 18: **End to end per answer**, prefill included (batch 1, one B300; mean seconds and generated tokens per answer over N problems; GSM8K: consecutive problem IDs 1–192; budgets 4,096 and 8,192 tokens). On GSM8K, prefill 4 / decode 2 spends $1.38\times$ the decode time per generated token of Original 2/2 (pooled time over pooled tokens), but its answers are shorter.

8K context, 32 steps	n=1	n=2	n=3	n=4
native n/n	834	342	200	125
Ours 4/n, one prompt cache	813	406	271	203
1K context, 2,048 steps		n=2		n=4
native n/n		586		219
Ours 4/n, one prompt cache		570		253

Table 19: **Decode throughput** of Ouro-2.6B (tokens/s, best over tested batch sizes, one B300): native model against the loop adapter prefilling with 4 loops with one stored prompt KV cache.

O Design Variants

Table 20 compares design variants. Training the same loop adapter with its maps frozen at the identity, so that a LoRA decoder reads the last prefill loop's KV directly, changes MMLU by at most 0.4 points and is not worse on GSM8K ($+2.3$ [0.2, 4.5] at prefill 4 / decode 1), but lowers RULER-16K by 11.4 points at prefill 4 / decode 1 and by 2.6–3.4 points at two decode loops. Restricting the key map to a RoPE-commuting scale, which is what allows one stored prompt KV cache, shows no significant difference against unrestricted key maps with one cache per loop in five tested configurations (RULER-16K; Table 20).

Other design variants. Sharing generated-token KV across decode loops ("shared answer KV", Appendix G) costs 2.4 to 3.0 GSM8K points (-2.5 [−4.5, −0.5], -2.4 [−4.2, −0.7], -3.0 [−4.8, −1.3] at 4/2, 3/3, 4/4). A gated branch over the first prefill loop's KV improves RULER-4K by 2.3 [1.0, 3.6] points at prefill 4 / decode 1 with no significant difference on GSM8K ($+0.5$ [−0.9, 2.0]).

variant	m/n	task	difference
maps frozen at identity†	4/1	RULER-16K	-11.4 [−14.7, −8.2]
	4/1	GSM8K	$+2.3$ [0.2, 4.5]
	4/1	MMLU	-0.2 [−0.6, 0.1]
	4/2	RULER-16K	-2.6 [−4.5, −0.7]
	4/2	GSM8K	$+0.2$ [−1.3, 1.7]
unrestricted keys‡	2/2	RULER-16K	-3.4 [−5.6, −1.3]
	1/1	RULER-16K	$+0.1$ [−1.3, 1.5]
	2/2	RULER-16K	$+0.6$ [−0.8, 2.0]
	3/1	RULER-16K	-0.9 [−2.9, 1.2]
	3/2	RULER-16K	$+1.2$ [−0.4, 2.7]
	4/2	RULER-16K	-1.0 [−2.6, 0.5]

Table 20: **Design variants** on Ouro-2.6B: variant minus our loop adapter at prefill m / decode n (full sets, paired). Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals. †Same data, budget, seed and (m, n) sampling; only the maps are not trained (LoRA only, the decoder reads the last prefill loop's KV directly). ‡Unrestricted key maps with one prompt KV cache per decode loop.

P Looped Decision Models: Setup and Checks

We evaluate on the 3,270 answerable questions of LevBench's public development set (never its locked test set), with intervals over record groups as Lev does. Each prefill count m is one continued training (seed 0); the main comparison (m equal to the backbone's loops) was repeated with three more seeds and against a control trained for twice as many updates. With all decision loops, the model with the most prefill loops is no better than the control on the Ouro backbones and below it on Lev-3B (-1.8 [−3.0, −0.7]); with $m = n$ the trained models are within a few points of the control in both directions (Table 21).

Setup and checks. Both arms continue each released Lev model for 140 updates on the 8,925-record public S31 training subset with Lev's hyperparameters, from the same weights. Our reference forward runs the full sequence through both the prefill and the decision loops, so its training computation is about twice the control's. Without training, more prefill loops do not help (-3.0 on Lev-1.4B at one decision loop), and the control itself moves little against the released models at one loop ($+0.5$ on Lev-1.4B, $+1.3$ on Lev-2.6B). Repeating both arms with other seeds, the one-loop gain ranges from $+3.9$ to $+5.3$ on Lev-1.4B and from $+5.1$ to $+7.5$ on Lev-2.6B, ev-

model	prefill m	decision 1	decision 2	decision 3	decision 4
Lev-1.4B	1	$+2.5^{*}$ [1.3, 3.7]	-2.8^{*} [−4.2, −1.4]	-6.1^{*} [−7.8, −4.6]	-9.6^{*} [−11.2, −7.9]
Lev-1.4B	2	$+2.9^{*}$ [1.3, 4.7]	$+2.6^{*}$ [1.4, 3.8]	-0.1 [−1.3, 1.1]	-2.7^{*} [−4.0, −1.4]
Lev-1.4B	3	$+5.7^{*}$ [3.9, 7.6]	$+3.8^{*}$ [2.3, 5.1]	$+1.8^{*}$ [0.7, 3.0]	-1.2^{*} [−2.4, −0.03]
Lev-1.4B	4	$+5.3^{*}$ [3.4, 7.2]	$+3.8^{*}$ [2.3, 5.3]	$+2.0^{*}$ [0.7, 3.2]	-0.1 [−1.3, 1.1]
Lev-2.6B	1	$+0.7$ [−0.5, 2.0]	-7.5^{*} [−9.1, −5.9]	-12.5^{*} [−14.1, −10.8]	-13.0^{*} [−14.7, −11.3]
Lev-2.6B	2	$+4.4^{*}$ [2.6, 6.2]	-0.8 [−2.0, 0.4]	-3.5^{*} [−4.8, −2.3]	-4.1^{*} [−5.5, −2.8]
Lev-2.6B	3	$+6.8^{*}$ [4.8, 8.7]	$+2.3^{*}$ [0.9, 3.8]	-0.4 [−1.5, 0.7]	-0.9 [−2.0, 0.3]
Lev-2.6B	4	$+7.5^{*}$ [5.7, 9.5]	$+3.5^{*}$ [2.0, 5.0]	$+0.5$ [−0.7, 1.7]	-0.3 [−1.4, 0.8]
Lev-3B	1	$+0.3$ [−0.7, 1.3]	-8.1^{*} [−9.7, −6.6]	–	–
Lev-3B	2	$+4.4^{*}$ [2.8, 6.1]	-1.8^{*} [−3.0, −0.7]	–	–

Table 21: **Lev decision models, both directions.** Accuracy of the model trained to prefill the context with m loops minus the matched-data and matched-update control (native n/n) at n decision loops, LevBench development set (3,270 answerable questions), in points, training seed 0 (other seeds in the text); brackets are 95% bootstrap intervals over record groups (*: interval excludes zero). Lev-3B's backbone has two loops. Above the diagonal ($m < n$) every difference is negative.

ery interval excluding zero (four seeds each); on Lev-3B it is $+4.4$, $+5.1$, $+4.0$ and $+0.9$ [−0.8, 2.7] (mean $+3.6$). A control trained for two epochs (twice the updates, about the training computation of the other arm) still trails it at one decision loop by $+4.0$ [2.1, 6.0] on Lev-1.4B, $+5.5$ [3.7, 7.4] on Lev-2.6B and $+4.0$ [2.3, 5.7] on Lev-3B.

Q Prefill Loops in a Hybrid Model

SenseNova-Loop-9B stacks three Gated-DeltaNet layers per full-attention layer. Its prompt state therefore has two parts: the attention KV, which the decoder attends to, and each Gated-DeltaNet layer's convolution and recurrent state, a fixed-size summary of the prompt from which the decoder continues and which it keeps updating with its own tokens (the prefill stops before the last prompt token, which the decoder processes first). We give the one-visit decoder the second visit's state for one kind of layer and the first visit's for the other (Table 22). The loop adapter was trained only with both states from the first or both from the second visit, so we also retrain it, with the same recipe and seed, giving the decoder the second visit's state only in the Gated-DeltaNet layers for prefill-2 samples (below). These are experiments on one checkpoint; they show how the decoder responds to each state, not where information is stored. In every condition the layers after the repeated block keep the two-visit prefill's states, so a comparison with the one-visit prefill changes those states too; only the comparisons among the three conditions hold them fixed.

Without training, both mixed-state conditions score far above the one-visit prefill (RULER-16K $+50.2$ and $+50.5$, against $+66.6$ for the full second-visit state; MMLU-Pro $+23.7$ and $+21.5$ against $+36.2$); these contrasts also change the post-block states, so they do not isolate either part. With the trained loop adapter the two parts behave differently. The condition with the second-visit Gated-DeltaNet state is higher on long-context scores: HELMET $+9.1$ [6.5, 11.6] and RULER-16K $+4.9$ [2.1, 7.7], against -0.2 and $+0.7$ for the second-visit attention KV, which lowers RULER-4K (-4.5 [−7.2, −2.0]). The mixed attention-KV condition is higher on MMLU-Pro ($+4.6$ [2.0, 7.2]; mixed Gated-DeltaNet condition $+2.0$ [−0.4, 4.5]). The ARC-Challenge loss of the full second-visit state (Table 23) appears with the Gated-DeltaNet part (-2.7 [−4.5, −0.9]; mixed attention-KV condition $+0.8$). In the "GDN state" condition the decoder still attends to the first visit's KV, consistent with exact retrieval relying on attention in hybrid models (Afendulev et al., 2026); what differs is the recurrent state it continues from (and the post-block states, see above). On Ouro, which has only attention layers, KV from more prefill loops likewise helps knowledge and loses exact recall (Appendix R). The same trained loop adapter, given the second-visit state only in its Gated-DeltaNet layers, scores above the full second-visit state on HELMET by 5.6 points and on RULER-4K/8K/16K by 6.5, 4.2 and 3.7, shows no detectable difference on ARC-Challenge, and scores below it on MMLU-Pro by 3.5. The loop adapter retrained on this mixed state confirms the pattern within its own training distribution: against its own prefill 1 / decode 1 it gains on HELMET ($+7.2$ [4.7, 9.7]) and RULER-16K ($+6.1$ [3.2, 9.1]) and loses ARC-Challenge (-2.8 [−4.7, −0.9]); against the original

	untrained one-visit decoder			trained loop adapter		
benchmark	attention KV	GDN state	both	attention KV	GDN state	both
RULER-4K	+41.3 [37.2, 45.5]	+48.0 [44.0, 52.0]	+61.9 [57.8, 65.8]	−4.5 [−7.2, −2.0]	+2.7 [−0.5, 6.0]	−3.8 [−7.6, 0.2]
RULER-8K	+49.0 [44.8, 52.9]	+49.6 [45.7, 53.7]	+64.2 [60.0, 68.3]	−2.6 [−5.1, −0.2]	+2.3 [−0.8, 5.6]	−1.8 [−4.8, 1.3]
RULER-16K	+50.2 [46.0, 54.1]	+50.5 [46.4, 54.6]	+66.6 [62.7, 70.5]	+0.7 [−1.4, 2.8]	+4.9 [2.1, 7.7]	+1.2 [−1.7, 4.0]
MMLU-Pro	+23.7 [20.3, 27.0]	+21.5 [18.4, 24.8]	+36.2 [32.8, 39.8]	+4.6 [2.0, 7.2]	+2.0 [−0.4, 4.5]	+5.5 [2.9, 8.1]
ARC-C	+5.2 [3.2, 7.3]	+7.7 [4.9, 10.4]	+9.6 [6.7, 12.5]	+0.8 [−0.6, 2.1]	−2.7 [−4.5, −0.9]	−2.9 [−4.8, −1.0]
HELMET	+32.1 [30.1, 34.1]	+37.4 [35.5, 39.3]	+51.8 [49.8, 54.0]	−0.2 [−2.7, 2.2]	+9.1 [6.5, 11.6]	+3.5 [0.8, 6.1]

Table 22: **Mixed prompt-state configurations after a two-visit prefill.** SenseNova-Loop-9B, one decode visit, after a two-visit prefill. Within the repeated block, "attention KV" gives the decoder the second visit's attention KV and the first visit's Gated-DeltaNet (GDN) convolution and recurrent states; "GDN state" the reverse; "both" the second visit's states throughout. Layers outside the block keep the two-visit prefill's states. Differences are to the one-visit prefill with the same decoder, in score points; brackets are paired 95% bootstrap intervals.

loop adapter at prefill 2 / decode 1 it is better on HELMET (+4.4 [2.7, 6.2]) and RULER-4K/8K/16K (+6.7, +2.8, +2.9) and worse on MMLU-Pro (−2.7 [−5.0, −0.3]). Which state to give the decoder is therefore a task-dependent trade-off that holds with and without training for it.

Table 23 is the SenseNova-Loop-9B counterpart of Figure 6, at one decode visit through the repeated block instead of two. With the same loop adapter, prefilling with two visits instead of one raises MMLU-Pro (+5.5), BBH (+7.0) and HELMET (+3.5), lowers ARC-Challenge (−2.9), and leaves GSM8K, MATH-500 and HumanEval within noise. Against native 2/2, prefill 2 / decode 1 shows no detectable difference on GSM8K (−0.6 [−1.6, 0.4]; the 0.59× answer time of Section D.4 is measured for prefill 1 / decode 1) and scores lower on the other benchmarks, most on HELMET (−16.8); giving the decoder the second-visit state only in the Gated-DeltaNet layers narrows that HELMET gap to −11.2 [−13.0, −9.4]. The native one-visit model scores far lower on every benchmark.

R Prompt Format and HELMET Categories

HELMET shows which kinds of long-context task depend on the prefill loops (Table 24). On both Ouro reasoning models, prefilling with four loops instead of one improves retrieval-augmented QA by 15 points and re-ranking by 13–18, but lowers exact recall from a JSON key–value store on Ouro-1.4B-Thinking (−23.5); against Original 4/4, Ours 4/2 has higher point estimates on retrieval-augmented QA and in-context learning (+6.1 [4.1, 8.4], +6.6 [3.0, 10.5]) and loses exact recall (58.8 vs. 100), so its HELMET average is lower (−8.7 [−13.3, −4.1]). SenseNova-Loop-9B does not follow the Ouro pattern on recall and re-ranking. The recall category is small (34 items on Ouro).

The effect of the prefill loops depends on the prompt format (Table 25). At one decode loop, more prefill loops give a negative but inconclusive estimate on RULER-4K, where answers are copied verbatim, and lower 4-shot MATH-500 on both model sizes; without exemplars the same comparison turns positive on two model sizes and two decode loop counts. The aligned adapter (Ours) behaves the same way: four prefill loops instead of one change 4-shot MATH-500 by −9.4 [−13.6, −5.4] and −18.0 [−22.2, −14.0] points at one and two decode loops, and 0-shot MATH-500 by +8.0 and +10.4. Removing exemplars also changes length and format, so we read this as an interaction with the prompt.

	score				prefill 2 / decode 1 minus	
benchmark	native 1/1	prefill 1 / decode 1	prefill 2 / decode 1	native 2/2	prefill 1 / decode 1	native 2/2
GSM8K	2.2	95.6	95.2	95.8	-0.4 [-1.4, 0.6]	-0.6 [-1.6, 0.4]
MATH-500	3.2	88.0	89.8	94.8	$+1.8$ [-1.0, 4.6]	-5.0 [-7.2, -2.8]
MMLU-Pro	18.9	60.3	65.8	68.2	$+5.5$ [2.9, 8.1]	-2.3 [-4.4, -0.2]
ARC-C	52.4	66.0	63.1	66.9	-2.9 [-4.8, -1.0]	-3.8 [-5.3, -2.4]
BBH	2.6	44.5	51.5	57.6	$+7.0$ [5.9, 8.0]	-6.1 [-7.0, -5.2]
HumanEval	3.0	75.0	76.2	81.7	$+1.2$ [-4.9, 7.3]	-5.5 [-11.0, 0.0]
HELMET	7.2	56.9	60.3	77.1	$+3.5$ [0.8, 6.1]	-16.8 [-18.7, -15.0]

Table 23: **SenseNova-Loop-9B at one decode visit.** One loop adapter serves prefill 1 / decode 1 and prefill 2 / decode 1; native 2/2 runs the repeated block twice per token (56 layer executions instead of 32). Same protocols as Table 15 in Appendix L (MMLU-Pro 980, HELMET 6,646, BBH 6,511 items). MBPP+ is omitted: its prompt forces the answer to start with code, and this thinking model then opens a reasoning block inside the code (native 1/1: 294, prefill 1 / decode 1: 236, prefill 2 / decode 1: 372, native 2/2: 375 of 378 answers), so its score measures this format failure. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

comparison	RAG	re-rank	ICL	recall
2.6B-T, prefill $4-1$	$+15.4$	$+13.4$	$+5.1$	-2.9
1.4B-T, prefill $4-1$	$+14.5$	$+18.0$	$+9.3$	-23.5
SenseNova, prefill $2-1$	$+1.2$	-32.9	$+21.6$	$+24.0$
2.6B-T, 4/2 − native 4/4	$+6.1$	-6.3	$+6.6$	-41.2

Table 24: **HELMET by category** (score differences on a 0–100 scale; prefill comparisons at one decode loop, same loop adapter). More prefill loops help retrieval-augmented QA (RAG), re-ranking and in-context learning (ICL) on Ouro but can lose exact key–value recall. Items: 700 / 41 / 333 / 34 per category on Ouro, 4,200 / 246 / 2,000 / 200 on SenseNova; intervals are in `paper_numbers.json`.

model	decode n	task	prefill 4 − prefill n
Ouro-2.6B	1	RULER-4K	-3.8 [-8.2, 0.3]
	1	MATH, 4-shot	-12.0 [-16.2, -7.8]
	1	MATH, 0-shot	$+8.0$ [4.2, 11.6]
	2	MATH, 0-shot	$+7.4$ [3.4, 11.4]
Ouro-1.4B	1	MATH, 4-shot	-4.4 [-8.4, -0.6]
	1	MATH, 0-shot	$+4.8$ [1.2, 8.4]

Table 25: **The effect of prefill loops changes with the prompt setup.** Same loop adapter, base models, prefill 4 / decode n minus prefill n / decode n; MATH-500 scored on the final answer. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

S Reproducibility

`python ouro/scripts/paper_numbers.py compute` recomputes the paired differences, intervals, means, timing ratios and decode throughput of this paper from the accompanying per-item records and compares them with expected values hard-coded in the script, which mirror the numbers printed here; only the MBPP+ format counts of Appendix H are reported as measured. Every run's command, configuration, checkpoint hash, log and outputs are in `ouro/results/provenance`. Re-running the main-table cells from scratch reproduces GSM8K and MMLU with at least 99.6% of item scores identical and RULER-16K with 94–100% identical and averages within 0.9 points (`ouro/results/verify_rerun.txt`). `REPRODUCE.md` maps every result to its scripts and gives the training and evaluation commands that regenerate the per-item records.